

Universidad ORT Uruguay

Facultad De Ingeniería

DOCUMENTACIÓN OBLIGATORIO DISEÑO DE APLICACIONES 1

Nicolas Bidenti (305108)
Santiago Canadell (282542)
Felipe Delgado (281987)

Tutor: Luis Simón

2025

https://github.com/IngSoft-DA1/305108_282542_281987

A handwritten signature in black ink, appearing to read 'Nicolás B.S.' in a cursive style.

Nicolás Bidenti Scarzela 27/04/2025

A handwritten signature in black ink, appearing to read 'Santiago Canadell' in a cursive style.

Santiago Canadell 27/04/2025

A handwritten signature in black ink, appearing to read 'Felipe Delgado' in a cursive style.

Felipe Delgado 15/05/2025

Índice

Descripción general	5
<i>Bugs conocidos</i>	5
Diagrama de Gantt	5
Dependencia Circular entre Tareas	6
Diseño	7
UML (todos los uml se encuentran en el zip)	7
Domain	7
All Domain	8
Exceptions	9
Ejemplo Domain Exception	10
Service	11
All Service	12
Exceptions	12
Ejemplo de Exception:	13
Models	14
Interfaces	15
Converters	15
Exporter	16
Data Access	17
All Data Access	18
Exceptions	19
Repositories	20
DIAGRAMA DE SECUENCIA	21
Diagrama de	21
Código	22
Estructura y Dependencias Generadas	22
Asignación De Responsabilidades	23
Explicación de validaciones de negocio	31
Mantenibilidad y extensibilidad	31
Git	33
Gitflow	33
Test	34
Cobertura De pruebas	34
Pruebas Funcionales De ABM	34
TDD	34
Buenas Prácticas (Mantenibilidad y extensibilidad)	39
Cálculo De Ruta Crítica	40
Diagrama de Gantt	42
Cambios en Entity Framework	43
Implementación de OnModelCreating	43

Agregación de IDs	43
Implementación de Interfaces	44
Algunos ejemplos de esto son	44
Implementación de controladores	45
Arquitectura y Responsabilidades	45
Beneficios En La Arquitectura	46
Integración y Escalabilidad	46
Exporter	46
IExporter	46
ExporterBase (Abstract)	47
CSVExporter - Implementación CSV	47
JSONExporter - Implementación JSON	47
Integración con LeaderPService	47
Ventajas para Extensibilidad	48
Cambios de DataAccess	48
Antes	48
Ahora (cambios implementados)	49
Migración a Entity Framework	50
Beneficios de los cambios	51
Análisis de Dependencias	51
Resolución de conflictos en asignación de recursos a tareas	51
Datos de Prueba:	53
Ejemplo de tarea con recursos:	54
Posibles Mejoras	54
Anexo	57
Cálculo De Ruta Crítica: TDD	57
Dependencias Externas	58
DHTMLX Gantt	58
Bootstrap	58
Full Calendar	59
Uso de IA (Primer Entrega)	59
Uso de IA (Segunda Entrega)	60
Foto coverage de pruebas	60

Descripción general

Empezamos el proyecto maquetando un UML que tenga sentido en base a la letra del obligatorio. En base a ese UML y la letra comenzamos a diagramar las partes principales del proyecto, dividiéndolas en packages que los cuales especificamos en profundo más abajo en la documentación. Comenzamos por Domain, luego seguimos con Data Access y finalmente Service para poder empezar con el Front-end.

El Front lo intentamos hacer intuitivo, armónico de ver y lo más fácil de usar posible para el usuario. Para eso usamos por ejemplo iconos de bootstrap y los errores en color rojo especificando porque estaba mal el camino que el usuario estaba tomando.

También hicimos que en el momento que a un usuario se le asigna un Rol, este se actualiza instantáneamente, sin necesidad de que el usuario haga logout y login para visualizar los cambios.

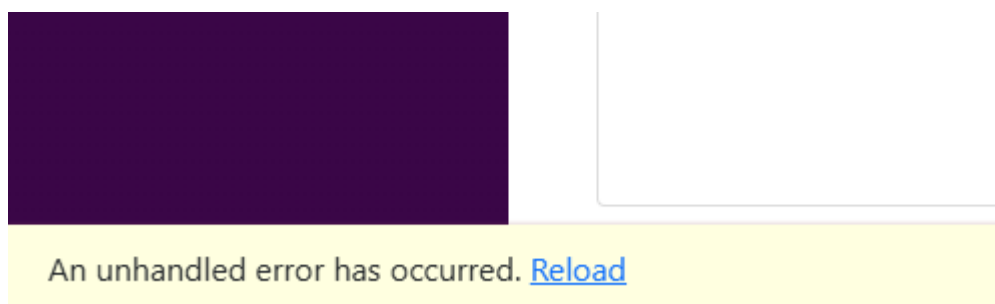
Además de estos aspectos, implementamos pruebas unitarias desde el principio, con el objetivo de garantizar la estabilidad y calidad del proyecto, permitiéndonos verificar los cambios que hacíamos en las funciones de forma rápida.

Con respecto al backend, decidimos estructurar la lógica del negocio de forma modular, separando las responsabilidades de cada componente dentro del servicio, como lo vimos en la clase de práctico. Esto no solo facilita la escalabilidad del proyecto, sino que también permite integrar nuevas funcionalidades sin afectar las partes ya implementadas.

Bugs conocidos

Diagrama de Gantt

Un bug que logramos encontrar en el programa es que a veces cuando entramos al Gantt Diagram y se elige el proyecto para visualizar el diagrama. Aparece en la parte inferior izquierda de la página que pide hacer un reload, al apretar en reload, se elige el proyecto a visualizar y se ve todo correctamente.



Sucede principalmente al entrar al diagrama de gantt, salir y entrar devuelta.

Donde creemos que está el error es en la parte del script de la page que usa la librería externa. Le dedicamos muchas horas a intentar solucionar este bug pero al no llegar a ninguna solución priorizamos seguir con las partes principales de la entrega ya que no es un bug que impida alguna funcionalidad o limite el uso del programa de una manera crítica.

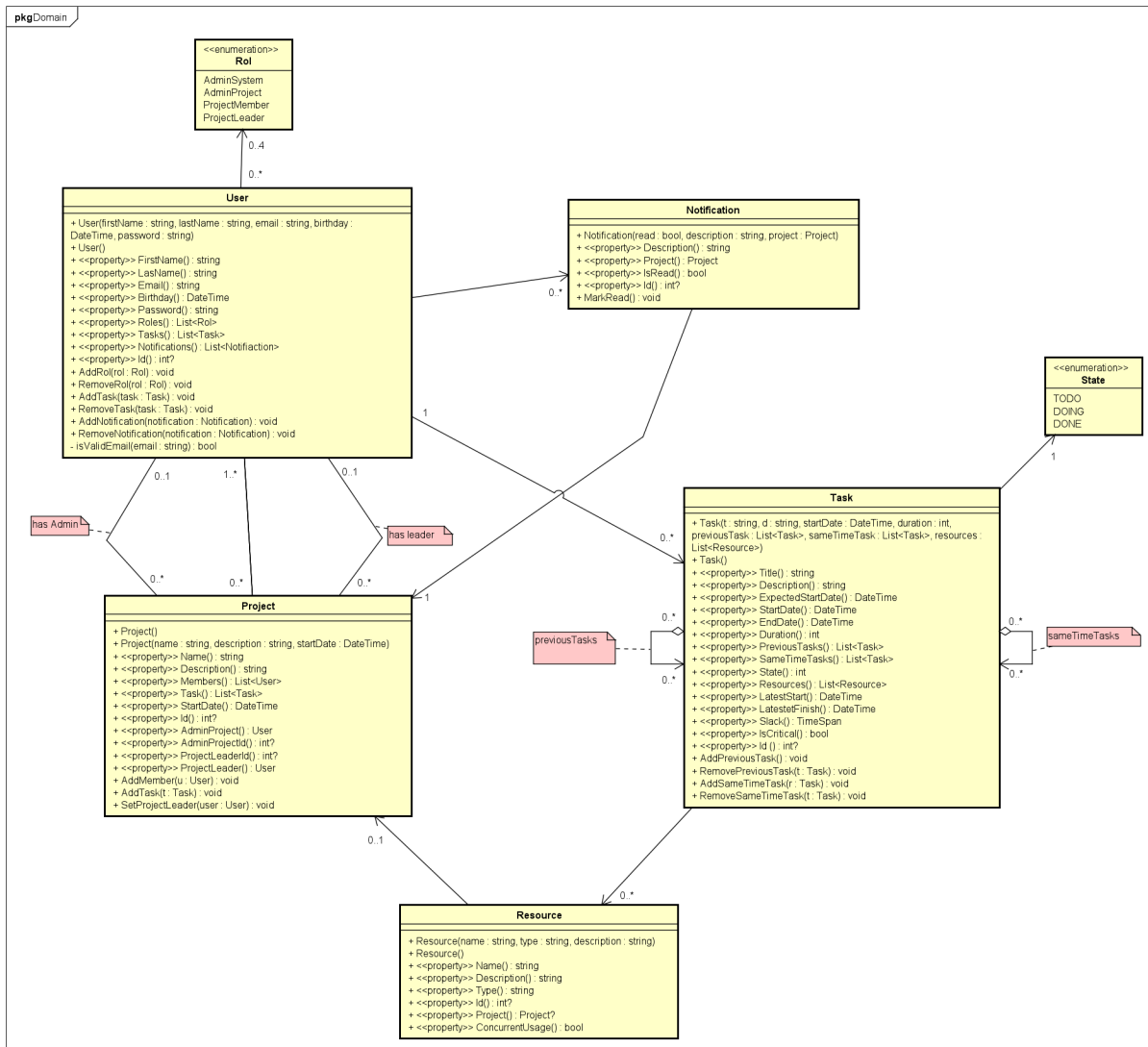
Dependencia Circular entre Tareas

Al hacer los últimas pruebas del sistema nos dimos cuenta que si creamos una tarea B que tenga dependencia de una tarea A y luego hacemos que A también tenga dependencia de B. Esto crea una dependencia circular que hace que no se muestre el diagrama de gantt. En sí este camino no tendría sentido que un usuario lo haga porque las dependencias van hacia solo un lado. Igualmente intentamos solucionarlo pero necesitábamos hacer recursión para traer todas las tareas previas y esto le agregaba mucha complejidad y no nos quedó suficiente tiempo para hacerlo.

Diseño

UML (todos los uml se encuentran en el zip)

Domain



Para las notificaciones como se tiene que enviar a todos los miembros de un proyecto, cada notificación tiene un proyecto. Para que al usuario le lleguen todas sus notificaciones lo controlamos del lado del service. Lo que hicimos es verificar si el proyecto tiene la notificación y si es el mismo proyecto que tiene el miembro, esa notificación es para ese usuario.

Para el caso de resource lo que hicimos fue controlarlo con los services , en el caso de que el recurso tenga proyecto ese recurso solo podrá ser utilizado por tareas de ese proyecto y si no tiene proyecto será un recurso global que puede ser utilizado por tareas de cualquier proyecto.

En task cuando se vincula con resource , si la task tiene habilitado el concurrent usage significa que puede ser utilizada por varias tareas, sin tomar en cuenta la restricción de la tarea con los recursos. Por otro lado, si este está deshabilitado, el resource no puede ser utilizado por varias tareas a la vez.

Para la relación entre tarea y estado, una tarea solo puede tener un solo estado asignado a la vez.

Para los roles, un usuario puede llegar a tener hasta cuatro roles en simultáneo.

Un proyecto puede o no tener project líder. Para inicializar un proyecto se necesita un usuario con rol de admin project pero una vez creado el proyecto, si el admin system elimina al admin project que creó al proyecto, el proyecto quedará sin admin.

Un punto a mejorar sería bajar el acoplamiento de la clase user ya que está directamente relacionada a la mayoría de clases de nuestro dominio. Esto porque es responsable de hacer muchos métodos , por ello también tiene baja cohesión, debería tener solo métodos relacionadas a user y no a las otras clases del dominio.

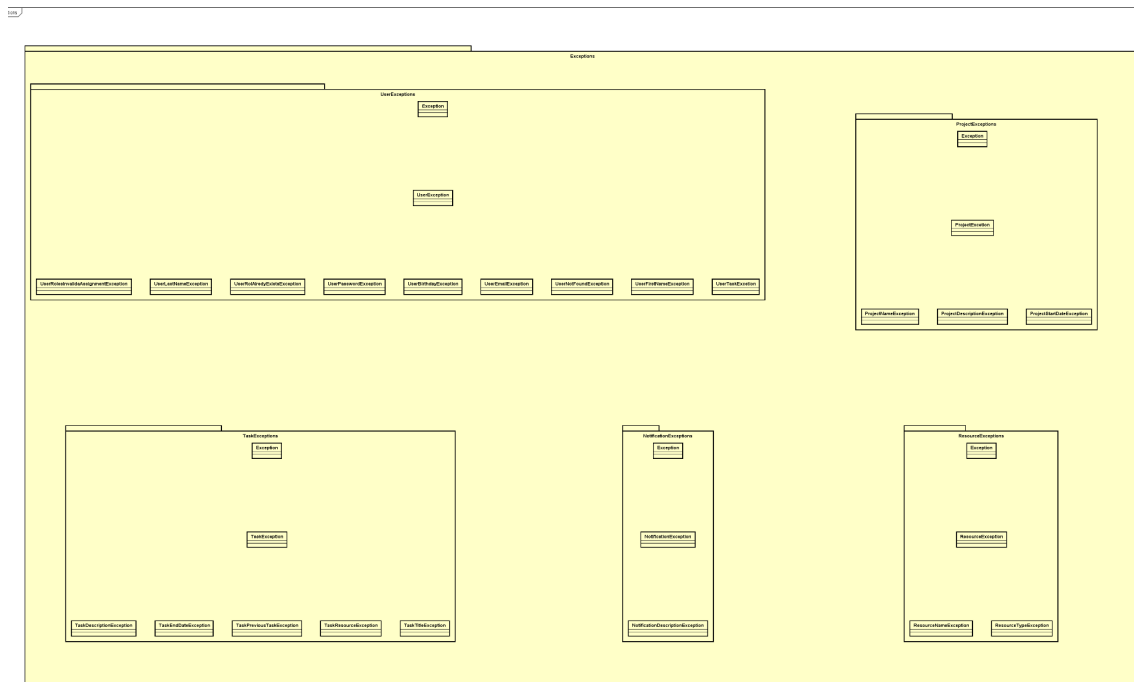
Para mejorar esto podríamos haber separado la clase user en varias clases que controlen las relaciones, por ejemplo hacer una clase intermedia llamada NotificationUser que gestione la lista de notificaciones del usuario como los métodos de agregar, eliminar, etc. De la misma forma con task.

All Domain

Se encuentra en el zip ya que es grande y no se logra ver en el docs.

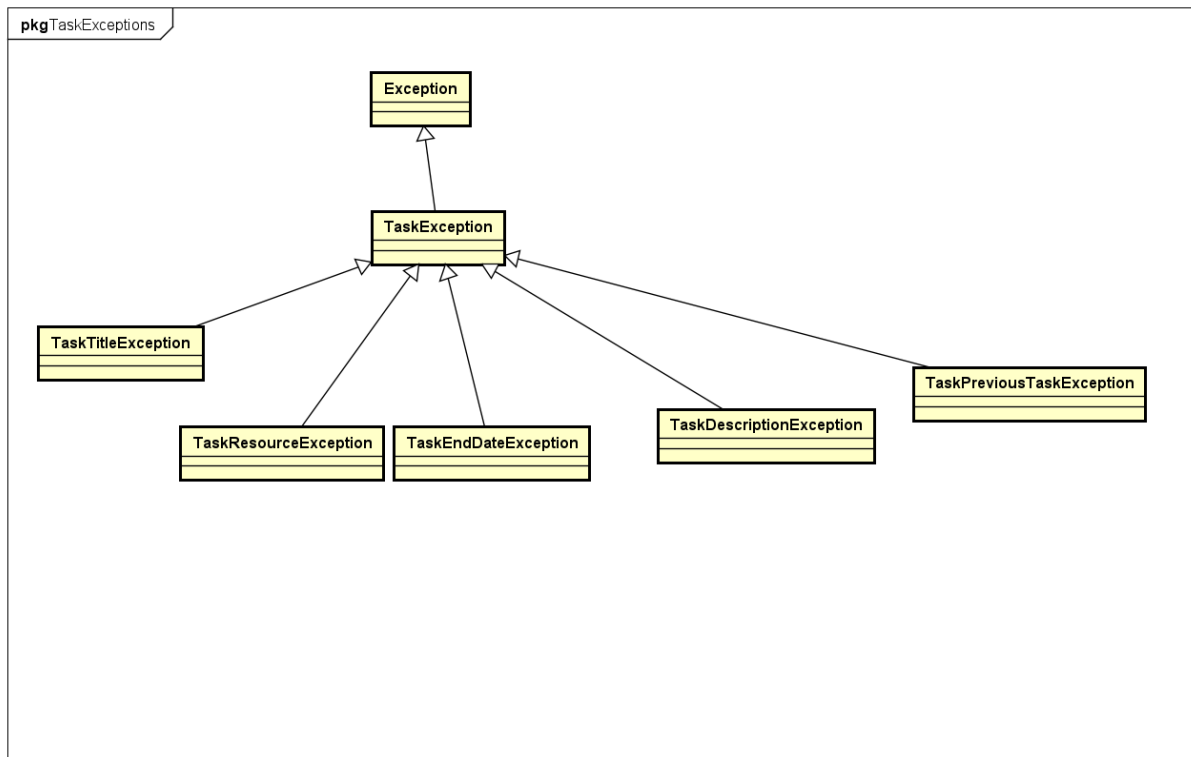
En el caso de all domain se muestra todo domain incluyendo las relaciones de sus clases con cada una de sus excepciones. En el diagrama quisimos mostrar la subdivisión de paquetes y cómo se relaciona cada uno.

Exceptions



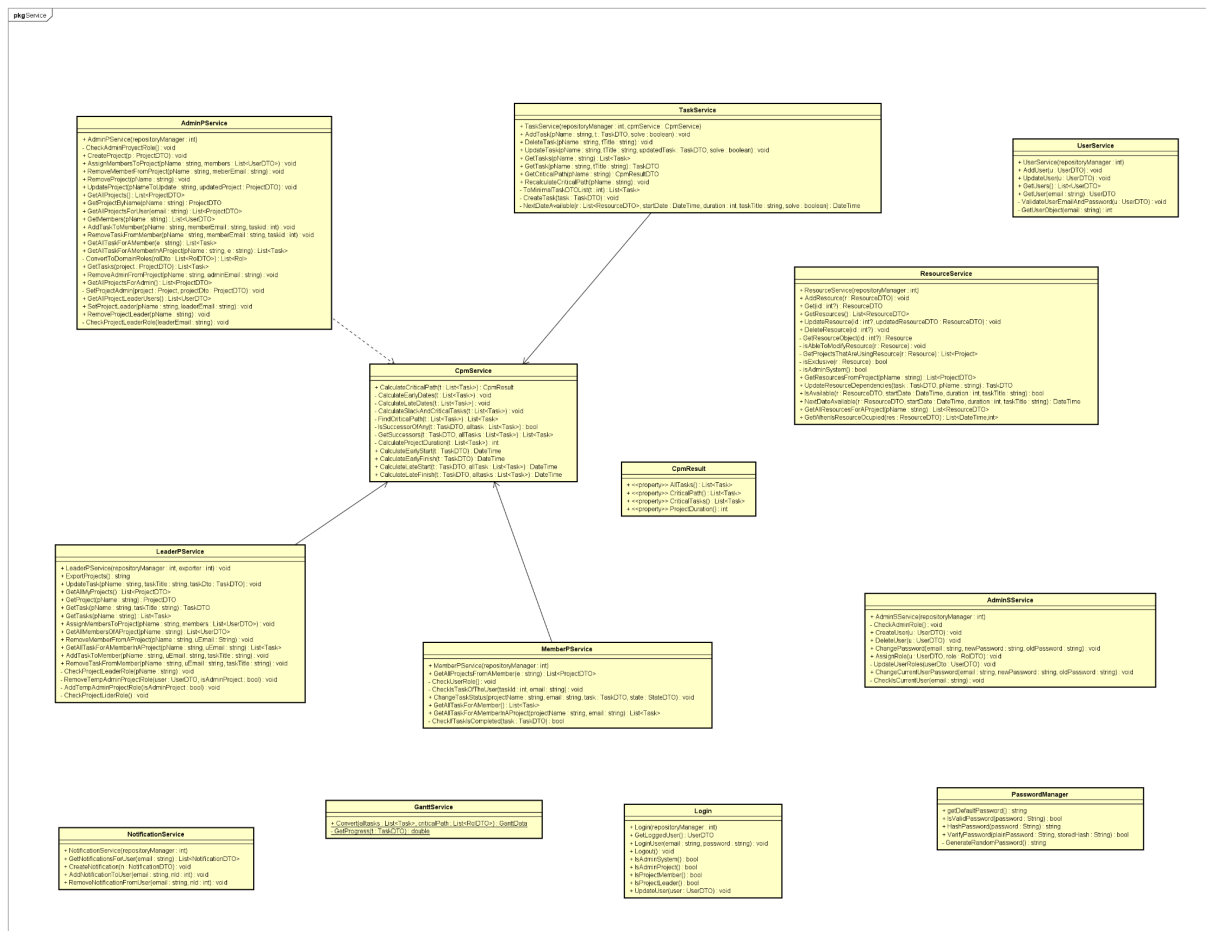
En este package se ven todas las excepciones usada para todos las clases de Domain,, luego cada uno de las entidades de domain tiene una exeption personalizada de la cual depende de este package.

Ejemplo Domain Exception



En el ejemplo de Domain Exception se subió la foto de un caso para mostrar cómo hicimos cada una de las exception de las entidades. Todas hereda de Exeption y tienen una exception padre con el nombre de la clase del dominio + "Exception". Luego esta tiene clases hijas que personalizan el mensaje de error.

Service



Este es el diagrama donde se encuentran todos los servicios, quisimos representar en este diagrama solo las relaciones que hay entre los services como tal, sin contar todo el resto. En general no existen dependencias ya que todo depende de interfaces para que no haya clases conectadas entre sí. En el caso de CPMSERVICE tuvimos que hacerlo así ya que si lo hacíamos con una interfaz se nos rompía la migración.

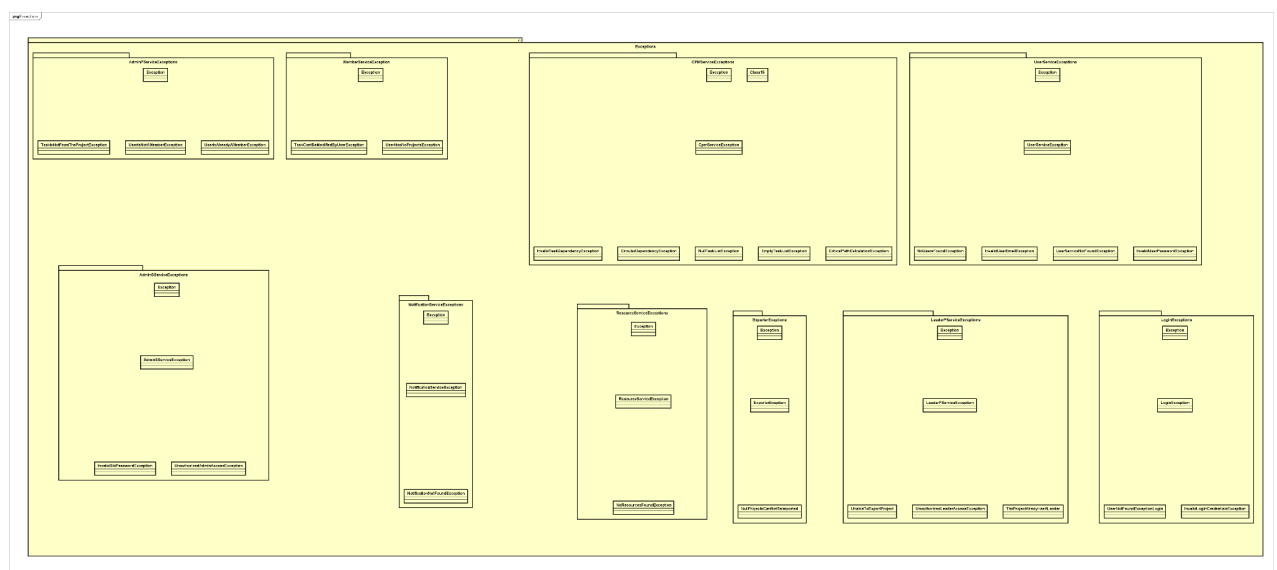
All Service

Se encuentra en el zip ya que es grande y no se logra ver en el docs.

En all service hicimos el diagrama en donde se ven todas las relaciones de service, desde los services , las exception, los models y todo el resto. En el caso de las exceptions de cada service y los models (los DTOs), preferimos no representar las relaciones en el diagrama ya que quedaba demasiado engorroso y entreverado.

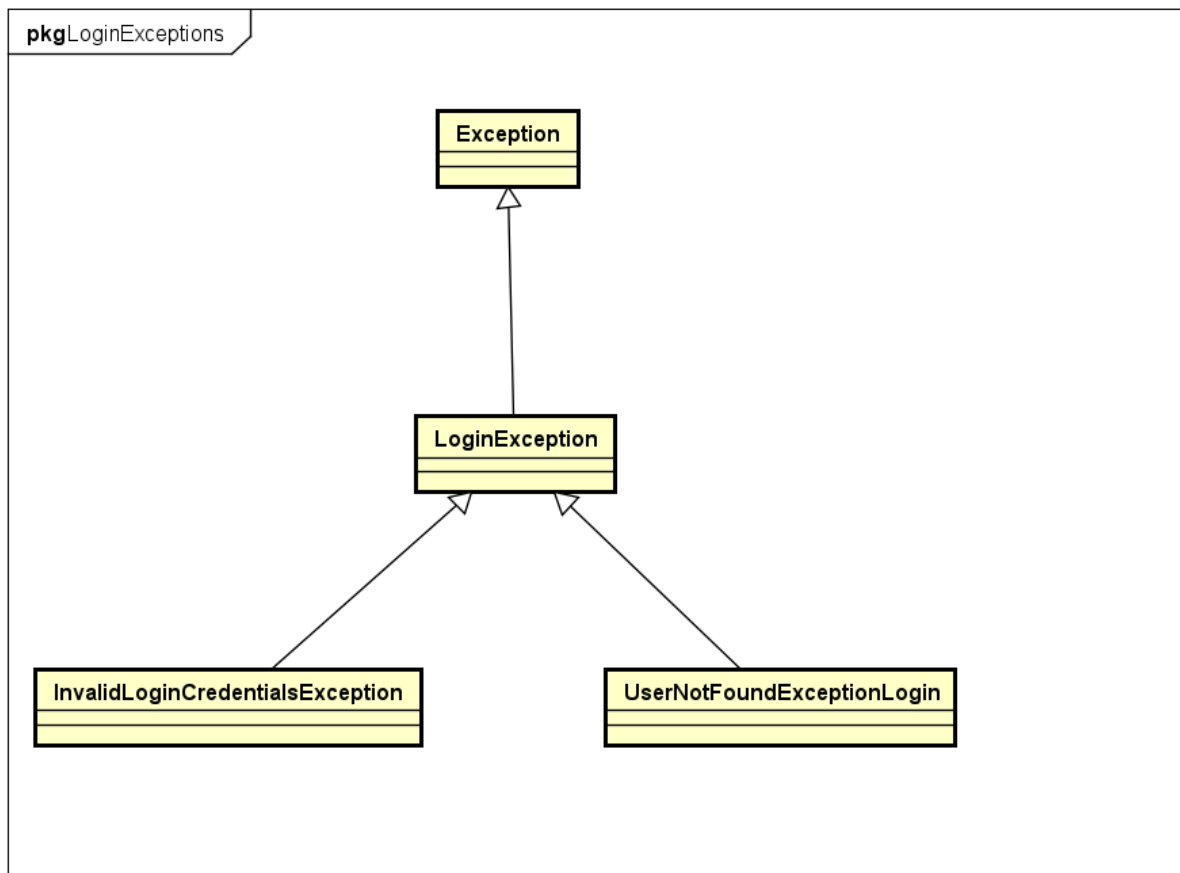
Tomar en cuenta que cada Service tiene una dependencia con el package models. Y también que cada Service tiene una dependencia con su propia excepción personalizada del package exception.

Exceptions



En este package se ven todas las excepciones usada para todas las clases de Service, como explicamos anteriormente, cada uno de los services tiene una excepción personalizada de la cual depende de este package.

Ejemplo de Exception:

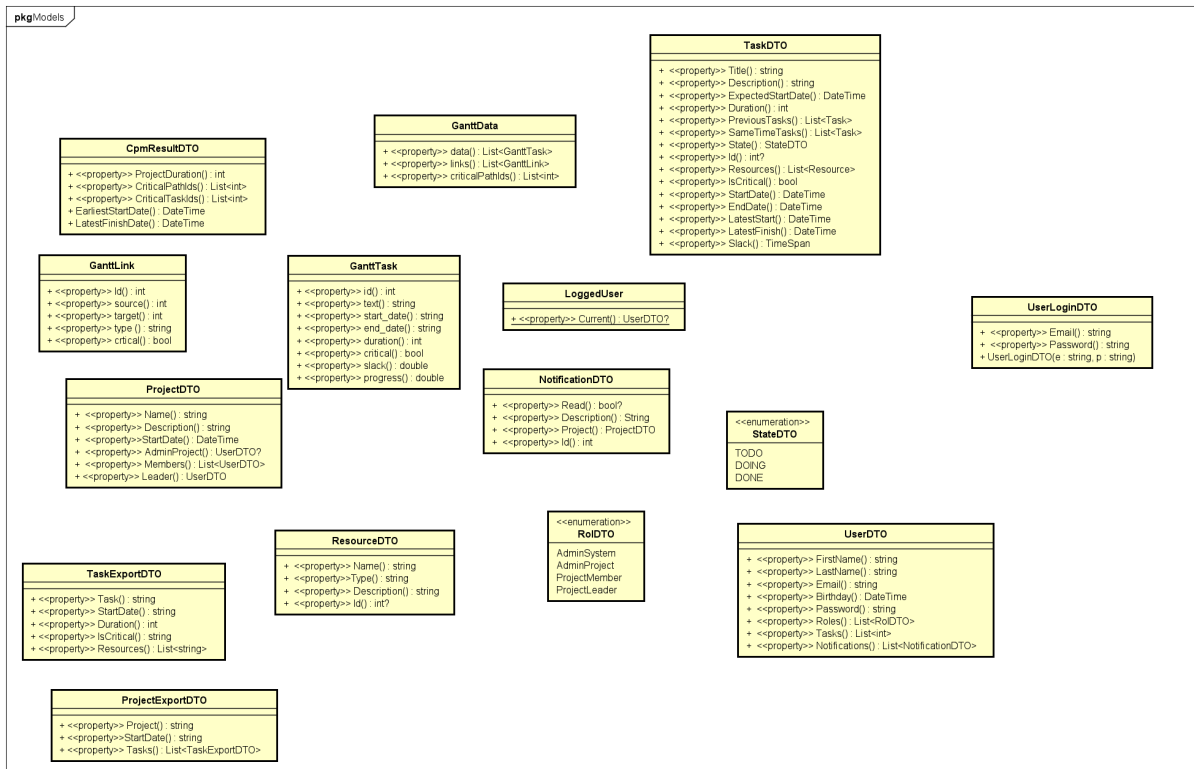


La clase exception es la clase raíz de todas las excepciones y esta tiene la funcionalidad básica para el manejo de errores.

Luego hicimos una clase llamada LoginException que hereda de exception y actúa como una excepción genérica para todos los problemas relacionados con el login. Es quien permite capturar cualquier error de autenticación de forma general

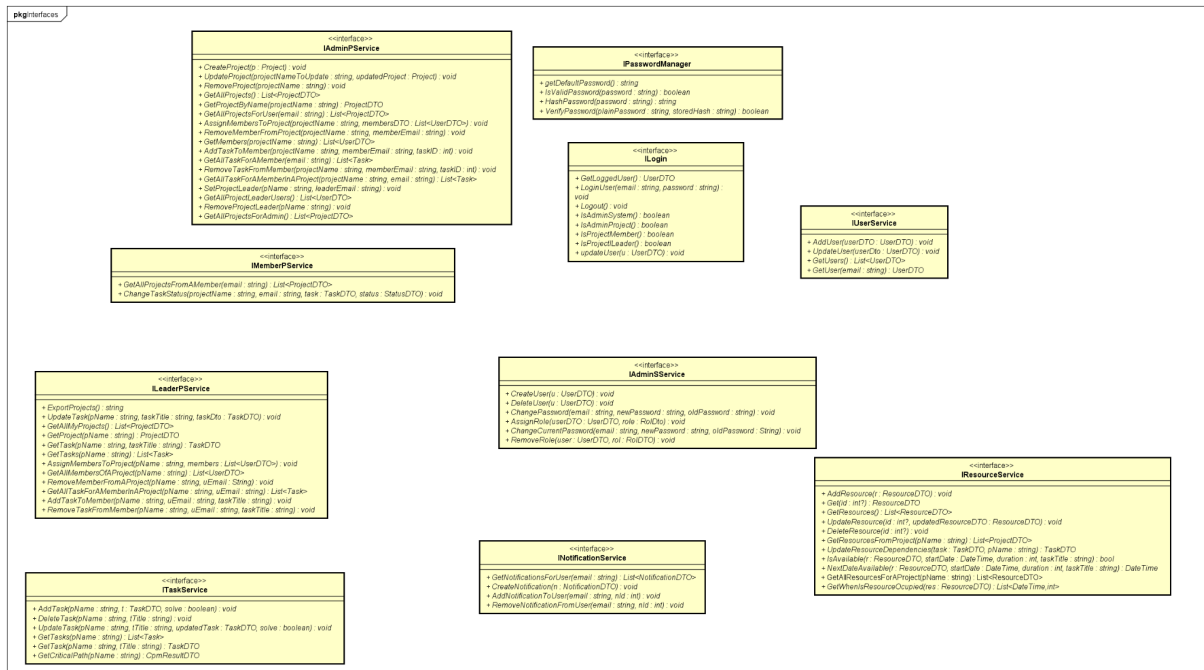
Luego por cada exception como por ejemplo InvalidLoginCredentialsException y UserNotFoundExceptionLogin estas heredan de LoginExceptions y personalizan el mensaje de error.

Models



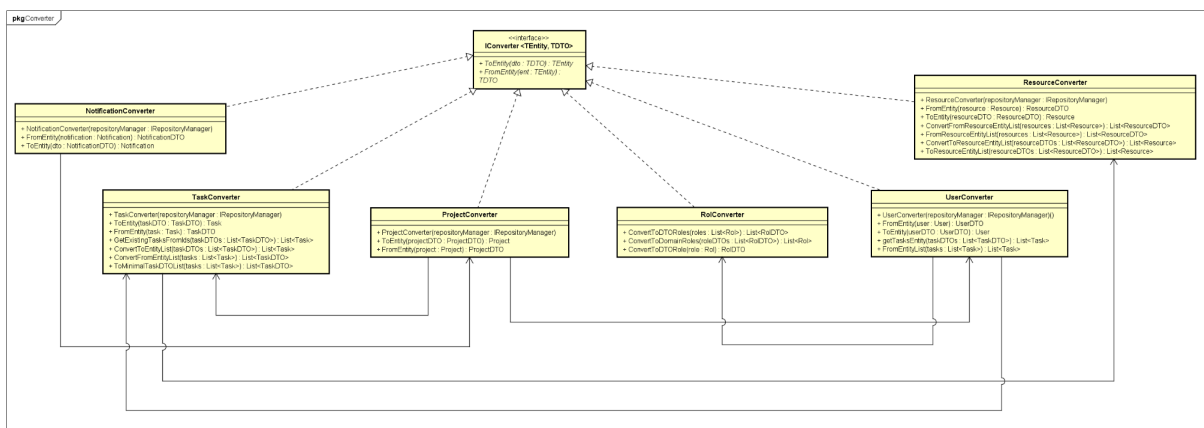
En este package se ven todas los DTOs usados para todos las clases de Service, como explicamos anteriormente, cada uno de los services tiene una dependencia con todo este package.

Interfaces



En este diagrama se muestran todas las interfaces que usamos en el Service para que las clases implementen las mismas, y que no dependan de clases concretas. A su vez, en estas definimos el comportamiento principal que debería tener cada uno de los servicios. En el diagrama de ALL SERVICE se muestra la vinculación de los servicios con las mismas.

Converters

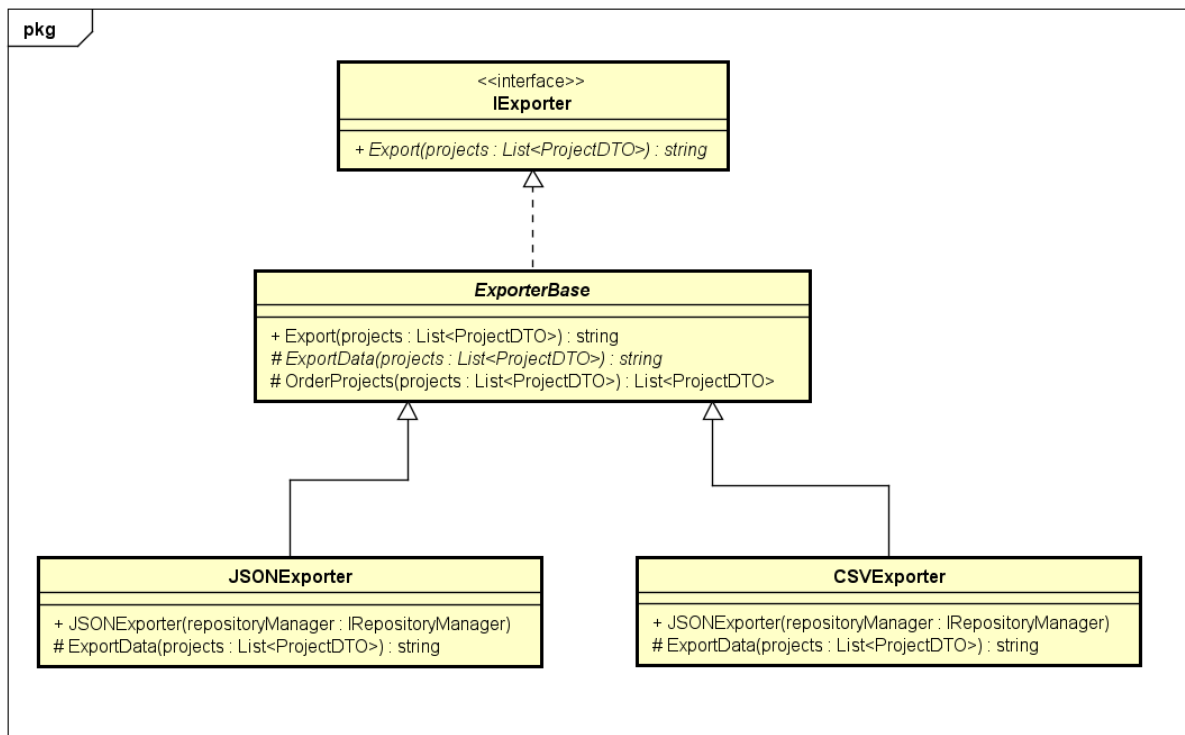


En este diagrama representamos las clases converter de cada entidad, para poder pasar de elemento del dominio a Ui como un DTO y viceversa, sin afectar la cohesión en nuestras clases del servicio.

Por cada entidad de dominio, la misma tiene un converter asociado. En cada converter, se implementa la interfaz IConverter, para la esta le especificamos la entidad concreta y su DTO. Esta Interfaz tiene las funciones básicas (FromEntity y ToEntity) que implementan todos los converters.

En el diagrama de all service se ve como los servicios usan estos converters según su necesidad.

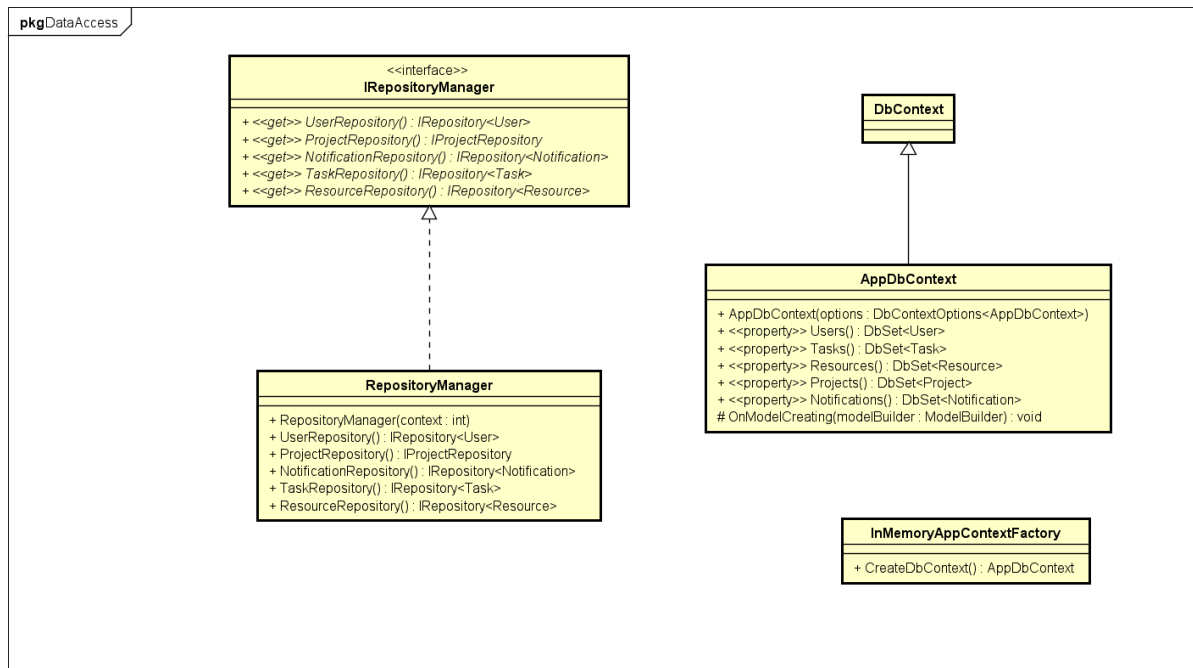
Exporter



En el diagrama hicimos que cumpla con el template method. Definimos el esqueleto de un en una clase base, dejando que las subclasses implementen los pasos específicos sin cambiar la estructura general.

Donde hay una interfaz que tiene un método Export y luego una clase Abstracta “ExporterBase” la cual implementa la interfaz. Esta clase tiene un metodo abstract ExportData el cual lo implementan ambos hijos, el JSONExporter y el CSVExporter. En all Service se ve la vinculación entre los servicios y exporter, donde LeaderPService tiene una dependencia con IExporter. Esta implemetnacion hace que sea muy fácil agregar nuevos tipos de exportadores como XMLExporter o PDFExporter sin tocar el código existente.

Data Access

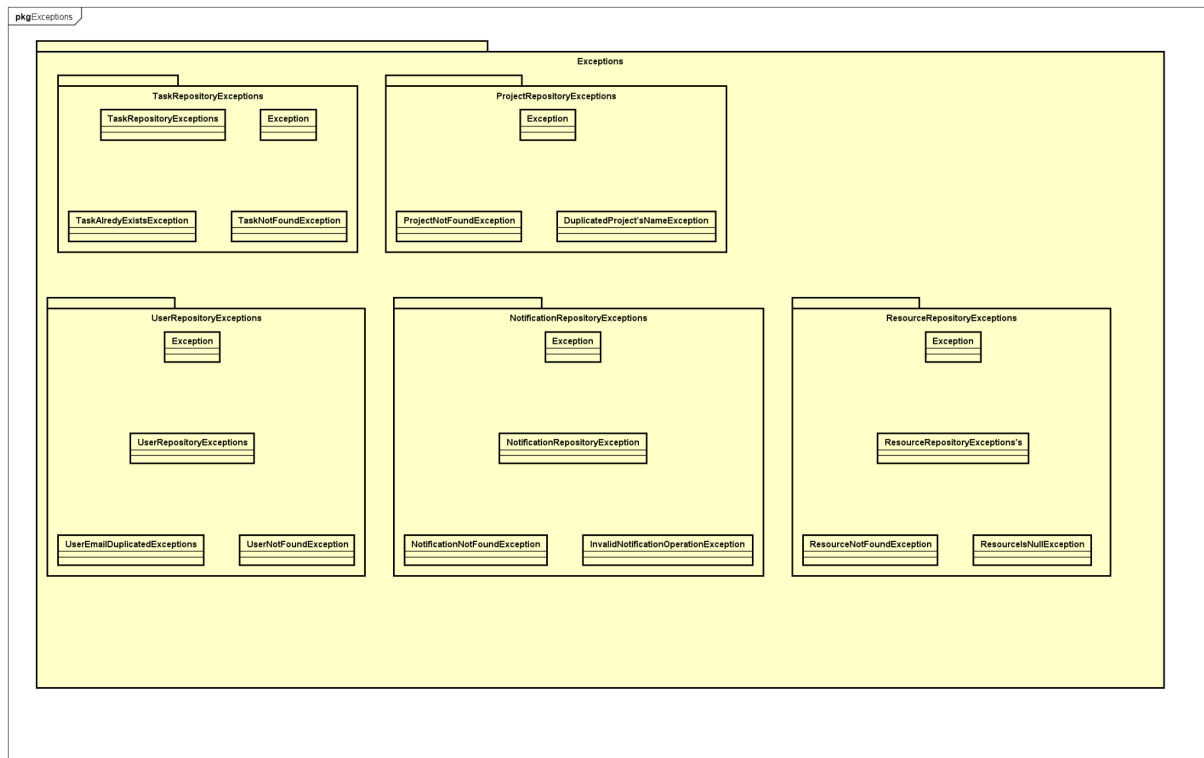


En este package se encuentra el manejador de repositorio, el cual se une mediante interfaces a los repositorios, específicamente mediante las interfaces de `IRepository` y `IProjectRepository`. Esta vinculación se logra ver en el Uml de All DataAccess.

Luego `AppDbContext` hereda de `DbContext` que es una clase base de Entity Framework que gestiona las entidades de la base de datos. Una de las partes importantes del `AppDbContext` es el `OnModelCreating` que es un método protegido que se usa para configurar el modelo de datos antes de que la base de datos sea creada o actualizada.

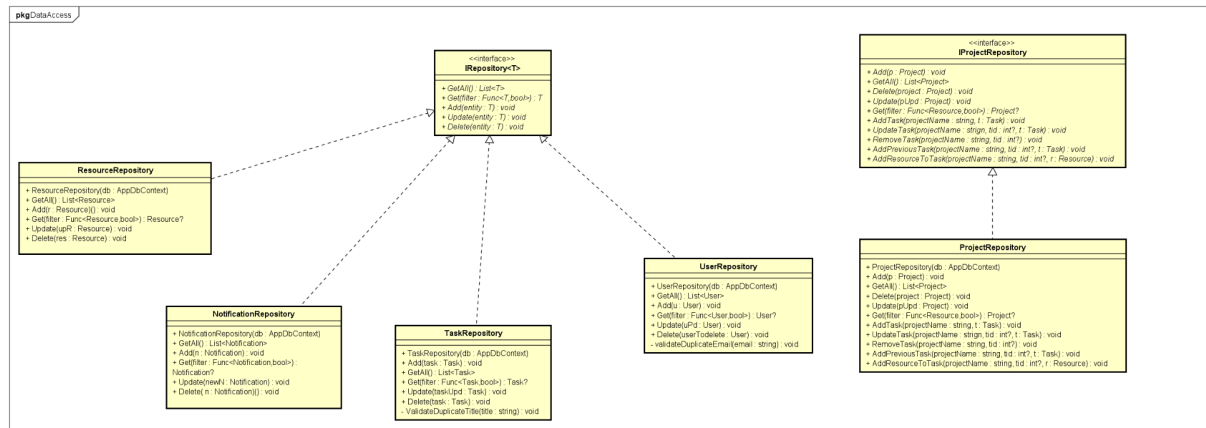
Full Text Access

Exceptions



Este Uml muestra todas las excepciones que hay en `DataAccess`, como explicamos arriba, los repositorios tienen dependencias de estas pero las flechas de dependencia no las hicimos quedaba muy cargado el Uml.

Repositories



Aqui se encuentran todos los repositorios con sus interfaces. En principio hicimos solo una interfaz (IRepository) la cual define el comportamiento de los repositories (add, delete, update, get, getAll), pero luego cuando la usabamos en RepositoryManager necesitabamos todos los metodos que contenia ProjectRepository que son mas de los que nos daba esta interfaz, entoces tuvimos que crear una interfaz unicamente para ProjecRepostiory (IProjectRepository) que tiene los metodos de IRepository + los metodos de manejo de tareas en los proyectos y demas.

DIAGRAMA DE SECUENCIA (todos los diagramas se encuentran en el zip)

Diagrama de Creación de recurso sin selección de Proyecto.

```
private void HandleValidSubmit()
{
    //Comenzamos desde aqui para: Crear Proyecto con asignacion de lider de proyecto
    try
    {
        if (string.IsNullOrEmpty(newProjectDTO.Name))
        {
            alertMessage = "Project name is required.";
            alertVisible = true;
            return;
        }

        if (string.IsNullOrEmpty(newProjectDTO.Description))
        {
            alertMessage = "Project description is required.";
            alertVisible = true;
            return;
        }

        _AdminProjectController.CreateProject(newProjectDTO);

        if (!string.IsNullOrEmpty(selectedProjectLeaderEmail))
        {
            _AdminProjectController.SetProjectLeaderToProject(newProjectDTO.Name, selectedProjectLeaderEmail);
            _AdminProjectController.UpdateProject(newProjectDTO.Name, newProjectDTO);
            successMessage = $"Project '{newProjectDTO.Name}' created successfully with {GetLeaderDisplayName(selectedProjectLeaderEmail)} as leader!";
        }
    }
    else
    {
    }
```

Diagrama de Resolución de conflictos en asignación de recursos a tareas

```
private async Task ForceCreateTask()
{
    // Comenzamos desde aqui: Resolución de conflictos en asignación de recursos a tareas
    try
    {
        _TaskController.AddTaskToProject(pendingProjectName, pendingTask, solved: true);
        taskSuccessMessage = "Task created (forced) successfully!";
        showForceConfirm = false;
        taskErrorMessage = string.Empty;

        await Task.Delay(1500);
        CloseCreateTaskModal(refreshData: true);
        Navigation.NavigateTo(Navigation.Uri, forceLoad: true);
    }
    catch (Exception ex)
    {
        taskErrorMessage = $"Error forcing task: {ex.Message}";
    }
}
```

Diagrama de Crear proyecto con asignación de líder de proyecto

```
private void HandleValidSubmit()
{
    //Comenzamos desde aqui para: Crear Proyecto con asignacion de lider de proyecto
    try
    {
        if (string.IsNullOrEmpty(newProjectDTO.Name))
        {
            alertMessage = "Project name is required.";
            alertVisible = true;
            return;
        }

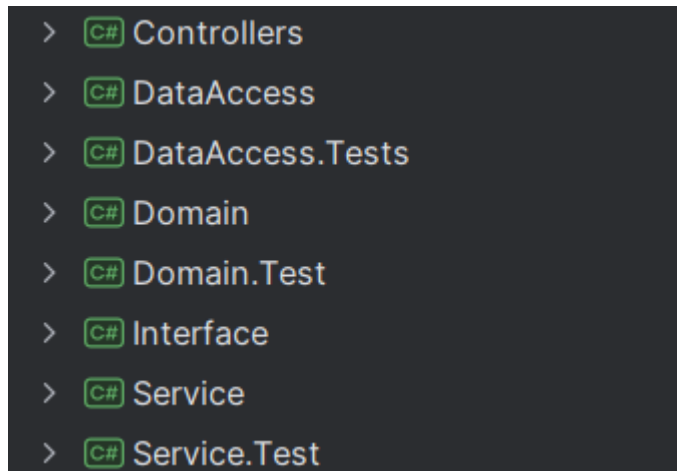
        if (string.IsNullOrEmpty(newProjectDTO.Description))
        {
            alertMessage = "Project description is required.";
            alertVisible = true;
            return;
        }

        _AdminProjectController.CreateProject(newProjectDTO);

        if (!string.IsNullOrEmpty(selectedProjectLeaderEmail))
        {
            _AdminProjectController.SetProjectLeaderToProject(newProjectDTO.Name, selectedProjectLeaderEmail);
            _AdminProjectController.UpdateProject(newProjectDTO.Name, newProjectDTO);
            successMessage = $"Project '{newProjectDTO.Name}' created successfully with {GetLeaderDisplayName(selectedProjectLeaderEmail)} as leader!";
        }
        else
        {
            successMessage = $"Project '{newProjectDTO.Name}' created successfully! You can assign a leader later.";
        }
    }
}
```

Código

Estructura y Dependencias Generadas



En el proyecto, la inyección de dependencias es muy importante para conectar las distintas capas del sistema, facilitando la interacción entre el front-end (UI) y el back-end. Para ello, en la capa de Interface (Front-End), se inyectan los servicios necesarios en cada página a través de la inyección de dependencias. Estos servicios, que están definidos en la capa Service, son responsables de la lógica de negocio y de interactuar con el dominio, pero sin que el front-end tenga acceso directo a las entidades del dominio.

Cuando una página necesita realizar alguna operación, como crear un proyecto o actualizar un usuario, obtiene el servicio mediante inyección, ya sea el Login, UserService, AdminPService, etc. Este patrón permite que cada página tenga acceso solo a los servicios necesarios, evitando la creación manual de instancias y ayudando a la reutilización del código.

Los servicios inyectados utilizan DTOs para trasladar los datos entre el front-end y el back-end. Por ejemplo, cuando se crea un proyecto, el front-end utiliza un ProjectDTO para enviar los datos estructurados al back-end, donde se transforman en una entidad de Project para ser procesados. De esta forma, el uso de DTOs separa la lógica del sistema de la representación de datos, lo que permite un mayor mantenimiento.

Asignación De Responsabilidades

DataAccess

Namespace	Clase	Responsabilidad
DataAccess.Repositories.UserRepository.cs	UserRepository.cs	Manejo de la lista de Users en la base de datos.
DataAccess.Repositories.ResourceRepository.cs	ResourceRepository.cs	Manejo de la lista de Resources en la base de datos.
DataAccess.Repositories.ProjectRepository.cs	ProjectRepository.cs	Manejo de la lista de Projects en la base de datos.
DataAccess.Repositories.NotificationRepository.cs	NotificationRepository.cs	Manejo de la lista de Notifications en la base de datos.
DataAccess.Repositories.TaskRepository.cs	TaskRepository.cs	Manejo de la lista de Tasks en la base de datos.
DataAccess.Repositories.UserRepository.cs	UserRepository.cs	Manejo de la lista de Users en la base de datos.
DataAccess.Exceptions.UserRepositoryExceptions	Carpeta de exceptions de user repository: -UserEmailsDuplicatedException.cs -UserNotFoundException.cs -UserRepositoryException.cs	Excepciones para casos border inválidos dentro de la clase UserRepository.
DataAccess.Exceptions.TaskRepositoryExceptions	Carpeta de exceptions que forma parte de project repository: -TaskAlreadyExistsException.cs -TaskNotFoundException.cs -TaskRepositoryExceptions.cs	Excepciones para casos border inválidos que forman parte de la clase ProjectRepository.
DataAccess.Exceptions.ResourceRepositoryExceptions	Carpeta de exceptions de resource repository: -ResourcesIsNullException.cs -ResourceNotFoundException.cs -ResourceRepositoryExceptions.cs	Excepciones para casos border inválidos que forman parte de la clase ResourceRepository.
DataAccess.Exceptions.ProjectRepositoryExceptions	Carpeta de exceptions de project repository: -DuplicatedProject'sNameException.cs	Excepciones para casos border inválidos que forman parte de la clase ProjectRepository.

	-ProjectNotFoundException	
.DataAccess.Exceptions.NotificationRepositoryExceptions	Carpeta de exceptions de notification repository: -InvalidNotificationOperationException -NotificationNotFoundException -NotificationRepositoryException	Excepciones para casos border inválidos que forman parte de la clase NotificationRepository.
DataAccess	AppDbContext.cs	Gestiona la conexión y acceso a la base de datos, y define las entidades User, Task, Resource, Project, y Notification
DataAccess	InMemoryAppContextFactory.cs	Crea una instancia de AppDbContext utilizando una base de datos en memoria para pruebas o desarrollo.
DataAccess	IRepositoryManager.cs	Define la interfaz para acceder a los repositorios de User, Project, Notification, Task, y Resource.
DataAccess	RepositoryManager.cs	Implementa la interfaz IRepositoryManager, gestiona la creación y acceso a los repositorios usando AppDbContext.

Service

Namespaces	Clases	Responsabilidad
Service	UserService .cs	Maneja operaciones de usuarios cómo agregar, actualizar y obtener usuarios. Valida correos y contraseñas, convierte entre objetos UserDTO y User, y gestiona

		roles y tareas de los usuarios.
Service	TaskService.cs	Maneja operaciones sobre tareas de proyectos, como agregar, eliminar, actualizar y obtener tareas, además de calcular y actualizar el camino crítico del proyecto.
Service	ResourceService.cs	Gestiona recursos, valida permisos de administrador para modificaciones, y maneja la conversión entre ResourceDTO y Resource
Service	PasswordManager.cs	Valida, hashea y verifica contraseñas. Utiliza SHA-256 para el hash, valida contraseñas con un patrón específico y proporciona una contraseña predeterminada.
Service	NotificationService.cs	Gestiona notificaciones en proyectos y usuarios. Permite obtener, agregar, eliminar y marcar como leídas las notificaciones, convirtiendo entre entidades Notification y NotificationDTO
Service	MemberPService.cs	Gestiona proyectos y tareas de miembros, validando roles y permisos para modificar tareas y acceder a proyectos.
Service	Login.cs	Gestiona el inicio y cierre de sesión, valida credenciales y roles del usuario.
Service	GanttService.cs	Convierte tareas y el camino crítico en datos para un diagrama de Gantt, calculando el progreso y creando enlaces entre tareas previas.
Service	LeaderPService.cs	Gestiona las tareas del líder de un proyecto, como obtener proyectos, asignar tareas a miembros, actualizar tareas y exportar proyectos. Asegura que el usuario tenga el rol adecuado (líder de proyecto) antes de realizar operaciones.
Service	CPMService.cs	Calcula las fechas de inicio y fin tempranas y tardías, el margen de tiempo holgura, y determina las tareas críticas. También maneja dependencias entre tareas y calcula la duración del proyecto.
Service	AdminSService.cs	Gestiona operaciones de administración del sistema, como crear, eliminar usuarios, cambiar contraseñas y asignar roles
Service	AdminPService.cs	Gestiona proyectos, miembros y tareas.

Service.Exceptions.UserServiceExceptions	Carpeta de exceptions del service user service: -InvalidUserEmailException -InvalidUserPasswordException -NoUsersFoundException -UserServiceException -UserServiceNotFoundException	Excepciones para casos border e inválidos dentro de la clase UserService.cs
Service.Exceptions.ExporterExceptions	Carpeta de exceptions del exporter service: -ExporterException -NullProjectsCanNotBeImported	Excepciones para casos border e inválidos dentro del Exporter.
Service.Exceptions.ResourceServiceExceptions	Carpeta de exceptions del service de resource service: -NoResourceFoundException -ResourceServiceException	Excepciones para casos border e inválidos dentro de la clase ResourceService.cs
Service.Exceptions.MemberServiceExceptions	Carpeta de exceptions del service de member p service: -TaskCantBeModifiedByUserException -UserHasNoProjectsException	Excepciones para casos border e inválidos dentro de la clase MemberService.cs
Service.Exceptions.LoginExceptions	Carpeta de exceptions del service de login service: -InvalidLoginCredentialsException -LoginException -UserNotFoundException	Excepciones para casos border e inválidos dentro de la clase LoginService.cs
Service.Exceptions.CPMServiceExceptions	Carpeta de exceptions del service de cpm service: -CircularDependencyException -CpmServiceException -CriticalPathCalculationException -EmptyTaskListException -InvalidTaskDependencyException -NullTaskListException	Excepciones para casos border e inválidos dentro de la clase CPMService.cs
Service.Exceptions.LeaderPServiceExceptions	Carpeta de exceptions del service de Leader service: -LeaderPServiceException -TheProjectAlreadyHasALeader -UnableToExportProject -UnauthorizedLeaderAccessException	Excepciones para casos border e inválidos dentro de la clase LeaderPService.cs
Service.Exceptions.AdminSServiceExceptions	Carpeta de exceptions del service de admin s service: -AdminSServiceException -InvalidOldPasswordException -CriticalPathCalculationException	Excepciones para casos border e inválidos dentro de la clase AdminSService.cs

	-UnauthorizedAdminAccessException	
Service.Exceptions.AdminPServiceExceptions	Carpeta de exceptions del service de admin p service: -AdminSServiceException -TaskIsNotFromTheProjectException -UserIsAlreadyAMemberException -UserIsNotAMemberException	Excepciones para casos border e inválidos dentro de la clase AdminPService.cs
Service.Models	Carpeta que contiene todos los dto de service: -CPMResultDTO -GanttData -GanttLink -GanttTask -LoggedUser -NotificationDTO -ProjectDTO -ResourceDTO -RolDTO -StateDTO -TaskDTO -UserDTO -UserLoginDTO -ProjectExportDTO -TaskExportDTO	Los archivos que se encuentran en la carpeta Models con el sufijo DTO representan objetos de transferencia de datos (Data Transfer Objects) que se utilizan para transportar datos entre capas del sistema.
Service.Interfaces	Carpeta que contiene todas las interfaces de service: -IAdminPService -IAdminSService -ILogin -IMemberPService -IPasswordManager -IResourceService -IUserService	Aseguran que las clases proporcionen implementaciones específicas para las operaciones necesarias, sin definir cómo se ejecutan esos métodos
Service.Converter	Carpeta que contiene los elementos de Converter: -IConverter -NotificationConverter -ProjectConverter -ResourceConverter -RolConverter -TaskConverter -UserConverter	Convierte las entidades del dominio (User, Task, Project, Resource y Roles) a sus representaciones en DTO y viceversa. Facilitan la transferencia de datos entre la capa de dominio y la capa de presentación o servicio.

Service.Exporter	Carpeta que contiene los elementos de Exporter: -IExporter -ExporterBase -CSVExporter -JSONExporter	Se encarga de exportar la información de los proyectos a diferentes formatos (como CSV y JSON). La clase base define el proceso de exportación, y las clases específicas (como CSVExporter y JSONExporter) implementan los detalles para exportar en el formato correspondiente.
------------------	---	--

Domain

Namespace	Clase	Responsabilidad
Domain.Exceptions.NotificationExceptions	Carpeta Notification.Exceptions: -NotificationException.cs -NotificationDescriptionException.cs	Excepciones para casos border e inválidos dentro de la clase Notification.cs
Domain.Exceptions.ProjectExceptions	Carpeta Project.Exceptions: -ProjectDescriptionException.cs -ProjectException.cs -ProjectNameException.cs -ProjectStartDateException.cs	Excepciones para casos border e inválidos dentro de la clase Project.cs
Domain.Exceptions.ResourceExceptions	Carpeta Resource.Exceptions: -ResourceException.cs -ResourceNameException.cs -ResourceTypeException.cs	Excepciones para casos border e inválidos dentro de la clase Resource.cs
Domain.Exceptions.TaskExceptions	Carpeta Task.Exceptions: -TaskDescriptionException.cs -TaskDurationException.cs -TaskEndDateException.cs -TaskException.cs -TaskPreviousTaskException.cs -TaskResourceException.cs -TaskTitleException.cs	Excepciones para casos border e inválidos dentro de la clase Task.cs

Domain.Exceptions. UserExceptions	Carpeta User.Exceptions: -UserBirthdayException.cs -UserEmailException.cs -UserException.cs -UserFirstNameException.cs -UserLastNameException.cs -UserPasswordException.cs -UserRoleAlreadyExistsException.cs -UserRoleNotFoundException.cs -UserRolesInvalidAssignmentException.cs -UserTaskException.cs	Excepciones para casos border e inválidos dentro de la clase User.cs
Domain	Notification.cs	Representar una notificación con un estado de lectura y una descripción.
Domain	Project.cs	Representa un proyecto con un nombre, descripción y fecha de inicio. Permite agregar miembros, tareas y notificaciones al proyecto.
Domain	Resource.cs	Representa un recurso con un nombre, tipo y descripción. Tiene una propiedad Id.
Domain	RolesEnum.cs	Define un Rol de los tres posibles: AdminSystem, ProjectMember, Admin Project y Project Leader.
Domain	State.cs	Define un State de los tres posibles: TODO, DOING y DONE.
Domain	Task.cs	Representa una tarea en un proyecto, con atributos de título, descripción, fechas de inicio y fin, duración, y estado. También deja la gestión de tareas previas y simultáneas, y recursos asociados
Domain	User.cs	Representa a un usuario con nombre, apellido, correo electrónico, fecha de nacimiento, contraseña, roles y tareas asignadas. Gestiona roles y tareas asociadas al usuario,

Controllers

Namespace	Clase	Responsabilidad
Controllers.AdminProjectController	AdminProjectController.cs	Gestiona las operaciones relacionadas con los proyectos, como crear, eliminar y actualizar proyectos. También maneja la asignación y eliminación de miembros y líderes de proyectos, como la asignación de tareas a miembros.

Controllers.AdminSystemController	AdminSystemController.cs	Gestiona las operaciones del sistema, como la creación y eliminación de usuarios, asignación de roles y gestión de contraseñas (cambio de contraseñas para usuarios específicos o para el usuario actual).
Controllers.GanttController	GanttController.cs	Gestiona las operaciones relacionadas con la visualización y cálculo del diagrama de Gantt y el camino crítico de las tareas. Sirve para obtener datos del diagrama de Gantt y calcular el camino crítico de un conjunto de tareas.
Controllers.LeaderProjectController	LeaderProjectController.cs	Gestiona las operaciones relacionadas con los proyectos para los líderes de proyecto, como asignar y eliminar miembros, asignar y eliminar tareas a los miembros, y obtener la lista de tareas y miembros de un proyecto. Además, deja exportar los proyectos en formatos JSON y CSV.
Controllers.LoginController	LoginController.cs	Gestiona las operaciones de inicio y cierre de sesión de los usuarios, así como la actualización de la información del usuario conectado.
Controllers.MemberProjectController	MemberProjectController.cs	Gestiona las operaciones relacionadas con los miembros de un proyecto, como cambiar el estado de las tareas asignadas a un miembro en un proyecto.
Controllers.ResourceAdminController	ResourceAdminController.cs	Gestiona las operaciones relacionadas con los recursos de un proyecto, como obtener los recursos de un proyecto, verificar cuándo está ocupado un recurso y calcular la siguiente fecha disponible para un recurso.
Controllers.ResourceController	ResourceController.cs	Gestiona las operaciones relacionadas con los recursos, como crear, actualizar, eliminar recursos y obtener la lista de recursos, tanto de manera general como por proyecto.
Controllers.SignUpController	SignUpController.cs	Gestiona el proceso de registro de un nuevo usuario, llamando al servicio correspondiente para agregar el usuario al sistema
Controllers.TaskController	TaskController.cs	Gestiona las operaciones relacionadas con las tareas dentro de un proyecto, como obtener tareas, agregar nuevas tareas, actualizar tareas existentes y eliminar tareas.
Controllers.UserController	UserController.cs	Gestiona las operaciones relacionadas con los usuarios, como obtener todos los usuarios y obtener la información de un usuario

	específico por su correo electrónico.
--	---------------------------------------

Explicación de validaciones de negocio

Las validaciones de negocio se implementaron dentro de los servicios, donde se maneja la lógica de negocio para asegurar la coherencia del sistema. Estas validaciones se asignaron a los servicios para centralizar la lógica y garantizar que las reglas se apliquen de manera uniforme, sin depender de otros componentes.

Ejemplo: En UserService, antes de agregar un usuario, se valida que el email no esté duplicado y que la contraseña cumpla con los requisitos de seguridad. Si alguna validación falla, se lanza una excepción para evitar registros incorrectos.

Mantenibilidad y extensibilidad

Uno de los aspectos clave para asegurar mantenibilidad y extensibilidad es la implementación de inyección de dependencias y el uso de DTOs para la transferencia de datos.

Un ejemplo es el siguiente:

Agregar un nuevo Rol a usuario.

Ej: quiero agregar el rol Manager

```
{  
  namespace Domain  
  {  
    81 usages  nbs282 *  4 exposing APIs  
    public enum Rol  
    {  
      AdminSystem,  
      ProjectMember,  
      AdminProject,  
      Manager // Nuevo rol añadido  
    }  
  }  
}
```


Luego se debería ajustar en UserDTO y en UserService.

```
private List<Rol> ConvertToDomainRoles(List<RolDTO> roleDTOs)
{
    var roles = new List<Rol>();

    foreach (var roleDTO in roleDTOs)
    {
        switch (roleDTO)
        {
            case RolDTO.AdminSystem:
                roles.Add(Rol.AdminSystem);
                break;
            case RolDTO.ProjectMember:
                roles.Add(Rol.ProjectMember);
                break;
            case RolDTO.AdminProject:
                roles.Add(Rol.AdminProject);
                break;
            case RolDTO.Manager: // Agregado el rol Manager
                roles.Add(Rol.Manager);
                break;
        }
    }

    return roles;
}
```

La UI no necesita realizar cambios significativos, ya que solo se han modificado los servicios y los DTOs, que continúan funcionando de manera consistente con el sistema.

La base de datos o el almacenamiento en memoria de usuarios simplemente necesita ser adaptado para manejar este nuevo rol sin afectar la estructura existente.

Cómo usamos servicios y DTOs desacoplamos la lógica de negocio del front-end, lo que facilita la modificación de funcionalidades sin alterar las demás partes del sistema. También agregar nuevos roles, funciones o métodos en los servicios, el impacto se limita solo a las capas correspondientes, sin necesidad de tocar todo el sistema.

Git

Gitflow

Para implementar Gitflow en nuestro proyecto, seguimos los pasos básicos que vimos en clase y los adaptamos a nuestras necesidades. Primero, nos aseguramos de tener las dos ramas base que son fundamentales en Gitflow: **develop** y **main**.

Luego, cada vez que comenzamos a trabajar en una nueva funcionalidad, creamos una rama a partir de develop. A cada rama le asignamos un nombre que refleja el tipo de tarea que estamos realizando. Por ejemplo, si estábamos trabajando en una nueva característica, la rama se creaba con el prefijo **feat/**, seguido de una descripción breve de la funcionalidad (ejemplo: feat/login).

Lo mismo ocurrió cuando teníamos que hacer correcciones, ahí usamos el prefijo **fix/** para las ramas de corrección de errores, como **fix/login-bug**, por ejemplo.

Una vez terminada la funcionalidad en cada rama, realizamos un merge de la misma de vuelta a develop. Esto nos aseguró que las nuevas características o correcciones se integren al flujo de trabajo de manera controlada y sin generar conflictos en main.

Es importante destacar que las ramas feature nunca interactúan directamente con main. En lugar de eso, creamos ramas release a partir de develop cuando estábamos listos para integrar las características y preparar una nueva versión.

Este flujo de trabajo nos ayudó a organizar mejor el desarrollo y asegurarnos de que todo el código estuviera en la rama correcta antes de pasar a producción, además de facilitarnos la colaboración entre nosotros.

Test

Cobertura De pruebas

Nos aseguramos de la calidad de nuestro sistema cumpliendo con el objetivo de tener al menos un 90% de cobertura de pruebas en todas las capas del backend, incluyendo dominio, data access y services.

Las pruebas de dominio, que cubren la lógica principal del proyecto, fueron hechas para garantizar que todas las funcionalidades críticas fueran verificadas. Lo mismo para las pruebas en las clases de Repository que manejan las listas de objetos del dominio.

Por otro lado service, que contienen la lógica para interactuar con otras capas y coordinar el flujo de la aplicación, también nos aseguramos de que estuvieran cubiertas al 90%.

Para lograr la cobertura de pruebas escribimos pruebas unitarias robustas utilizando TDD (Test-Driven Development) en cada uno de estos componentes. Lo que nos permitió verificar que cada función individual cumpliera con su propósito, asegurándonos de que las clases que interactúan con la “base de datos” y los servicios fueran probadas.

Cabe destacar que las partes del frontend, como las clases con ramas que tienen el sufijo -ui y -pov, no fueron cubiertas con pruebas de backend, ya que están relacionadas con la interfaz de usuario, pero sí fueron manejadas dentro del marco de desarrollo de la interfaz.

De esta forma, garantizamos que tanto el dominio, los servicios, como las capas de acceso a los datos estuvieran correctamente testeadas y cubiertas.

La foto del coverage de las pruebas se encuentra en el Anexo.

Pruebas Funcionales De ABM

- <https://youtu.be/D0FBIK4BS7Q>

TDD

Seguimos el ciclo de Red-Green-Refactor, que nos permitió mantener un desarrollo organizado y asegurar que cada funcionalidad estuviera probada antes de implementarla.

Primero siempre hicimos el RED. Donde implementamos únicamente el test para una nueva funcionalidad sin preocuparnos por la implementación que hiciera que la prueba pasará. El objetivo aquí era asegurarnos de que el test fallara, confirmando que no teníamos esa funcionalidad implementada aún. Una vez hecho esto, realizábamos el primer commit con el mensaje -m "red: información del test".

Después, pasábamos a la fase GREEN, donde implementábamos lo mínimo necesario para que el test pasara. Esto significaba escribir el código justo lo suficiente para que el test fuera exitoso. Al completar esta fase, realizábamos un nuevo commit con el mensaje -m "green: información de la implementación".

Finalmente Refactor, donde, tras haber visto el código funcionando, nos dábamos cuenta de que a veces podíamos mejorar la implementación. Esto podía a veces nos pasaba en el mismo momento de la implementación o después de un tiempo al revisar el código. En esta fase, si detectábamos oportunidades de mejora, hacíamos un refactor en el código, manteniendo su funcionalidad intacta (pasando la prueba como antes), pero optimizándolo. Este paso lo marcábamos con el commit -m "refactor: descripción de la mejora".

Este ciclo de Red-Green-Refactor lo aplicamos de forma consistente en cada una de las ramas del backend.

También implementamos excepciones personalizadas en lugar de utilizar excepciones genéricas como `ArgumentException`.

Lo que nos permitió un mejor control sobre los errores que pueden aparecer en las distintas capas del backend, tirando mensajes de error más claros y específicos.

Un ejemplo de esto es la clase `ProjectException`, que sirve como base para otras excepciones más específicas dentro del contexto del dominio de proyectos. Les mostramos un ejemplo de cómo la hicimos:

```
namespace Domain.Exceptions
{
    [3 usages] [3 inheritors] nbs282
    public class ProjectException : Exception
    {
        [3 usages] nbs282
        public ProjectException(string message) : base(message) { }

        nbs282
        public override string ToString()
        {
            return $"ProjectException: {this.GetType().Name} - {this.Message}";
        }
    }
}
```

A partir de esta excepción base, creamos excepciones más específicas como la `ProjectNameException`, que maneja el error cuando el nombre de un proyecto no es válido

```
namespace Domain.Exceptions
{
    [2 usages] nbs282
    public class ProjectNameException : ProjectException
    {
        [1 usage] nbs282
        public ProjectNameException()
            : base("Project name cannot be null, empty, or whitespace.") { }
    }
}
```

Estas excepciones personalizadas son fáciles y más claras de probar lo que nos facilitó el ciclo de **TDD**, en lugar de poner `ArgumentException` a todo podemos distinguir cual es excepción del nombre, de la descripción o de cualquier otro atributo de la clase.

Cada una de las excepciones personalizadas fue probada como parte de los tests unitarios, para asegurarnos de que se lanzarán de manera correcta en los escenarios adecuados y que su manejo fuera el esperado.

Aquí dejamos un ejemplo de TDD hecho en Gestión de Tareas con Duración y Dependencias, específicamente en la parte de Definición de Tareas.

Green: LatestStart implemented  SantiCanadell committed 3 days ago
red: LatestStart not implemented  SantiCanadell committed 3 days ago
Commits on May 5, 2025
green: TaskResourceException implemented  SantiCanadell committed last week
red: TaskResourceException not implemented  SantiCanadell committed last week
green: TaskPreviousTaskException implemented  SantiCanadell committed last week
red: TaskPreviousTaskException not implemented  SantiCanadell committed last week
fix: id attribute in task  SantiCanadell committed last week
update taskDomain  SantiCanadell committed last week
green: implementation of Id_WhenSet_ThenIdsAssigned  SantiCanadell committed last week
red: implementation of id  SantiCanadell committed last week

Nosotros utilizamos un Id como clave primaria, y es la manera de identificar los tasks.

red: implementation of id



SantiCanadell committed last week

```
Domain.Test/TaskTest.cs
@@ -139,9 +139,23 @@ public void State_WhenSet_ThenStateIsAssigned()
139 139      }
140 140
141 141
142 +      [TestMethod]
143 +      public void Id_WhenSet_ThenIdIsAssigned()
144 +      {
145 +
146 +          int expectedId = 123;
147 +          List<Task> previousTasks = new List<Task>();
148 +          List<Task> sameTimeTasks = new List<Task>();
149 +          Task task = new Task("Title", "Description", DateTime.Now, 1, previousTasks, sameTimeTasks);
150
151 +
152 +          task.Id = expectedId;
153
154 +
155 +          Assert.AreEqual(expectedId, task.Id);
156 +      }
157
158 +
159
160     }
161 }
```

En el primer commit se logra ver como se crea el test, sin implementar la Id en [Task.cs](#)

green: implementation of Id_WhenSet_ThenIdIsAssigned



SantiCanadell committed last week

```
1 file changed +1 -1 lines changed
Domain/Task.cs
@@ -96,7 +96,7 @@ public void RemoveSameTimeTask(Task task)
96 96      }
97 97      public State State { get; set; }
98 98
99 -
99 +      public int? Id {get; set;}
100 100     }
101 101
102 102 }
```

En el segundo commit se ve como luego del red se implementa la funcionalidad en el Domain [Task.cs](#) y este ahora pasa la prueba con lo mínimo e indispensable.

Buenas Prácticas (Mantenibilidad y extensibilidad)

Organización del Proyecto y Responsabilidades de cada Package

Organizamos el proyecto en diferentes packages para asegurar una estructura clara y comprensible. Dividimos el proyecto en los siguientes paquetes:

- **Domain:**

Este paquete contiene las entidades principales del dominio de la aplicación. Las clases dentro de este paquete representan los elementos centrales del sistema, como la clase Task, que encapsula las propiedades y comportamientos relacionados con las tareas que los usuarios gestionan. La lógica de negocio se encuentra en este nivel, asegurando que el modelo del dominio esté desacoplado de las otras capas.

- **Service:**

En este paquete se incluye la lógica de negocio que interactúa con las entidades del dominio. Los servicios ofrecen operaciones más complejas y funcionalidades específicas que requieren la manipulación de los objetos del dominio. También pueden incluir reglas de negocio y validaciones que permiten reutilización del código en otras capas de la aplicación.

- **DataAccess:**

Se encarga de gestionar la persistencia y recuperación de los datos, interactúa con la base de datos. Se asegura de que las entidades del dominio se puedan guardar y cargar correctamente desde el almacenamiento, garantizando una separación clara entre la lógica de negocio y el acceso a los datos.

- **UI:**

El paquete de interfaz de usuario (UI) contiene los componentes visuales de la aplicación. Su responsabilidad principal es presentar la información al usuario y capturar sus interacciones, tales como la creación, edición y eliminación de tareas.

Este paquete se comunica mediante controladores con la capa de Services para poder acceder a las funciones necesarias de cada página.

- **Controllers:**

Los controladores gestionan las solicitudes provenientes de la interfaz de usuario (UI) y coordinan la comunicación con los servicios de negocio. Su función principal es asegurar que las operaciones solicitadas por el usuario sean procesadas correctamente, delegando la lógica de negocio a los servicios correspondientes. Esto facilita la extensibilidad y mantenibilidad del proyecto, ya que cada componente tiene un rol bien definido dentro del sistema.

Cada una de estas capas tiene su correspondiente package de tests. Lo hicimos para mantener una separación clara de responsabilidades y facilitar la escalabilidad del proyecto, asegurando que cada capa estuviera correctamente testeada y sin dependencias circulares.

Uso de TDD (Test-Driven Development)

Nos permitió mantener un desarrollo organizado y garantizar que todas las funcionalidades fueran probadas antes de ser implementadas.

El ciclo de Red-Green-Refactor fue seguido rigurosamente para cada nueva funcionalidad.

Excepciones Personalizadas

Además, implementamos excepciones personalizadas en lugar de utilizar excepciones genéricas como `ArgumentException`. Nos sirvió para tener un control sobre los errores que podrían ocurrir en las diferentes capas del backend, proporcionando mensajes de error más específicos para cada tipo de excepción.

Por ejemplo, creamos excepciones como `TaskTitleException` y `TaskResourceException` para manejar errores relacionados con las tareas del proyecto.

Convenciones de Nombres

Seguimos las convenciones de nombres de C#, utilizando CamelCase para las variables privadas (por ejemplo, `_title`, `_description`) y PascalCase para las propiedades públicas (por ejemplo, `Title`, `Description`).

Cálculo De Ruta Crítica

Hicimos un servicio para el cálculo de la ruta crítica basado en el método CPM (Critical Path Method), desarrollado en la clase `CpmService`. Lo que hace es analizar las tareas de un

proyecto, calcular sus fechas tempranas , tardías, y determinar las tareas críticas, las que no pueden retrasarse sin afectar la duración total del proyecto. La lógica de CPM se ejecuta mediante diversas funciones que se encargan de procesar las tareas del proyecto, estas son las funciones principales que implementamos:

1. Fechas Tempranas:

- La función `CalculateEarlyDates(List<Task> tasks)` calcula las fechas más tempranas de inicio (`StartDate`) y finalización (`EndDate`) para cada tarea. Si una tarea no tiene tareas anteriores devuelve la fecha esperada (`ExpectedStartDate`). Si tiene tareas anteriores, la fecha de inicio será la fecha de finalización de la tarea más tardía de los predecesores.

2. Fechas Tardías:

- La función `CalculateLateDates(List<Task> tasks)` calcula las fechas más tardías de inicio (`LatestStart`) y finalización (`LatestFinish`) para cada tarea sin afectar la duración total del proyecto. Empieza con las tareas finales (que no tienen tareas sucesoras) y luego se recorren las demás tareas en orden inverso, hacemos esto para asegurar que ninguna tarea sucesora se programe después de su fecha más tardía.

3. Slack (Holgura) y Tareas Críticas:

- La función `CalculateSlackAndCriticalTasks(List<Task> tasks)` calcula la holgura (`Slack`) de cada tarea. El `Slack` es la diferencia entre la fecha de inicio más tardía y la fecha de inicio más temprana. Si el `Slack` de una tarea es igual a cero o cercano a cero, hacemos que la tarea se marque como crítica. Son aquellas que su demora afectaría directamente la fecha de finalización del proyecto.

4. Ruta Crítica:

- La función `FindCriticalPath(List<Task> tasks)` determina la ruta crítica del proyecto. Esta ruta está formada por las tareas críticas, las que no tienen `Slack` o tienen un `Slack` muy bajo. La ruta crítica se construye iterando sobre las tareas críticas y uniando las tareas sucesoras.

5. Duración del Proyecto:

- La función `CalculateProjectDuration(List<Task> tasks)` calcula la duración total del proyecto, que es la diferencia entre la fecha de inicio más temprana de las tareas y la fecha de finalización más tardía. Esta duración nos sirve

para determinar los plazos de entrega y las metas del proyecto.

El cálculo se hace al ejecutar el método `CalculateCriticalPath(List<Task> tasks)`, que invoca estas funciones y maneja la lógica de cálculo. Este método devuelve un objeto `CpmResult` que contiene:

- **AllTasks:** Todas las tareas del proyecto con sus respectivas fechas, Slack y estados de criticidad.
- **CriticalPath:** La lista de tareas que forman la ruta crítica del proyecto.
- **CriticalTasks:** Las tareas críticas, las que tienen Slack igual a cero.
- **ProjectDuration:** La duración total del proyecto en días.

Este servicio facilita la integración de la información de la ruta crítica con otras herramientas de visualización, como los diagramas de Gantt, para ofrecer una representación gráfica del progreso del proyecto y sus tareas más importantes.

Diagrama de Gantt

Se implementó un diagrama de Gantt utilizando la librería `DHTMLX Gantt` para visualizar de forma gráfica las tareas de cada proyecto y sus dependencias. Además permite mostrar el camino crítico de forma gráfica, permitiendo identificar de forma clara las tareas cuya demora afecta directamente la duración total. La lógica se basa en el cálculo CPM (Critical Path Method), desarrollado en la clase `CpmService`, que analiza las fechas tempranas, tardías y la holgura de cada tarea para determinar cuáles son críticas. Estas tareas se convierten a formato JSON compatible con la librería mediante la clase `GanttService`, y se renderizan en una página Blazor (`Gantt.razor`) usando JavaScript. El gráfico incluye todas las tareas de un proyecto, mostrando su progreso, inicio, duración y relación con otras. Además al darle a un botón permite la visualización del camino crítico resaltando las tareas que lo componen con color rojo. La integración funciona completamente offline gracias a la inclusión local de los archivos `dhtmlxgantt.js` y `dhtmlxgantt.css`.

Cambios en Entity Framework

Implementación de OnModelCreating

Dentro de ApplicationDbContext tuvimos que agregar el método OnModelCreating.

Este se utiliza para configurar cómo las entidades del dominio, como User, Project, Task, Resource y Notification, se mapean a las tablas de la base de datos en Entity Framework. Definimos las claves primarias y especificamos las propiedades requeridas para varias entidades, como FirstName, LastName, Email, Birthday y Password en User. Además, configuramos conversiones personalizadas para campos como Roles (un List<Rol>) y State (un enum), que se almacenan como cadenas o ints en la base de datos y se convierten nuevamente cuando se recuperan.

También configuramos las relaciones de muchos a muchos entre entidades, como entre User y Notification, Project y User, y Task y Resource, utilizando tablas intermedias como UserNotifications y ProjectMembers.

Definimos el comportamiento de eliminación, ya sea en cascada o sin acción, dependiendo de la relación. Además, establecimos la relación entre Project y su AdminProject, así como la relación entre Task y otros elementos como PreviousTasks y SameTimeTasks.

Básicamente este método configura las reglas de mapeo, relaciones y conversiones de las entidades a la base de datos, y nos deja que Entity Framework maneje correctamente las operaciones de persistencia.

Agregación de IDs

Como Entity Framework maneja las entidades basándose en identificadores (ID).

Tuvimos que asegurarnos de que cada entidad tuviera su propia clave primaria para evitar ambigüedades y permitir un acceso correcto a los datos. Al definir clases como User, Project o Task, Entity Framework asume que debe existir una propiedad que represente esta clave primaria. Si no especificamos una propiedad explícita, por convención, busca una propiedad llamada Id o el nombre de la clase seguido de "Id" (por ejemplo, UserId para una entidad User). Esto es algo que agregamos para asegurar que cada registro tenga un valor único. Y nos permitió manejar correctamente las relaciones entre las tablas, como las claves foráneas (ProjectId, UserId), y poder insertar, actualizar y eliminar registros. Sin este identificador único, no podríamos diferenciar los registros ni gestionar las relaciones entre las entidades y además de esto se nos rompían todos los tests.

Implementación de Interfaces

Para mejorar la arquitectura y extensión de nuestro proyecto comparada a la entrega anterior, tuvimos que agregar interfaces para que las clases no interactúen directamente entre sí, sino que lo hicieran a través de estas interfaces. Esto permitió que cada clase solo tuviera acceso a la información que realmente necesita, logrando un diseño más limpio y flexible, donde las clases están desacopladas y pueden ser modificadas o reemplazadas sin afectar a las demás. Este enfoque cumple principalmente con el Principio de Abierto/Cerrado (OCP), ya que las clases están diseñadas para ser abiertas a la extensión pero cerradas a la modificación, permitiendo agregar nuevas implementaciones sin alterar el código existente, y con el Principio de Inversión de Dependencias (DIP), donde las clases dependen de abstracciones (interfaces) en lugar de clases concretas, reduciendo el acoplamiento entre módulos y facilitando la adaptación de nuevas implementaciones sin afectar al resto del código.

Algunos ejemplos de esto son

Definimos la interfaz `IRepositoryManager` para establecer las propiedades necesarias para acceder a los repositorios de diferentes entidades, como `UserRepository`, `ProjectRepository`, `NotificationRepository`, `TaskRepository` y `ResourceRepository`, brindando una forma centralizada y organizada de interactuar con los datos. La clase `RepositoryManager` implementa esta interfaz y actúa como un punto único de acceso a todos los repositorios, manteniendo referencias privadas a cada uno de ellos. El constructor de `RepositoryManager` recibe un `AppDbContext`, que utilizamos para crear los repositorios y permitir la interacción con la base de datos a través de Entity Framework.

Otra interfaz que hicimos es la de `IRepository` que define las operaciones básicas que implementamos en todos los repositorios para gestionar las entidades en la base de datos. Estas operaciones incluyen métodos (`GetAll`), obtener un único registro que cumpla con una condición específica (`Get`), agregar un nuevo registro (`Add`), actualizar un registro existente (`Update`) y eliminar un registro (`Delete`). El objetivo de esta interfaz es proporcionarnos un conjunto de métodos genéricos para interactuar con las entidades de manera consistente y desacoplada. En las implementaciones concretas de los repositorios, como `UserRepository`, `ProjectRepository`, `NotificationRepository`, `TaskRepository` y `ResourceRepository`, adaptamos estos métodos a las necesidades específicas de cada entidad. Por ejemplo, en `UserRepository`, el método `Add` valida si el correo electrónico del usuario ya está registrado antes de agregarlo, mientras que en `TaskRepository`, nos aseguramos de que no haya títulos de tarea duplicados.

Los controladores, que interactúan directamente con el frontend, dependen de interfaces que definen un contrato que los servicios deben cumplir. Estas interfaces nos proporcionan un conjunto claro de métodos para realizar operaciones relacionadas con las entidades de la

aplicación. Por ejemplo, interfaces como IUserService, IAdminPService o ILogin definen métodos específicos para gestionar usuarios, proyectos, roles y autenticación. Gracias a estas interfaces, los controladores pueden interactuar con los servicios sin tener que conocer los detalles de su implementación, lo que nos ayuda a mantener la separación de responsabilidades y facilitar la escalabilidad. Esto asegura que los controladores se mantengan livianos y enfocados únicamente en manejar las solicitudes y respuestas, mientras que los servicios gestionan toda la lógica detrás de las operaciones.

Implementación de controladores

Hicimos controladores como capa intermedia entre el frontend y la lógica de negocio, siguiendo el patrón de arquitectura en capas. Esta implementación garantiza que la interfaz de usuario nunca interactúe directamente con los servicios de negocio, manteniendo una separación clara de responsabilidades que facilita considerablemente el mantenimiento del sistema.

Arquitectura y Responsabilidades

Cada controlador se especializa en un dominio específico del sistema, actuando como un punto de entrada único para todas las operaciones relacionadas con ese dominio. Los controladores hacen las llamadas a los servicios correspondientes y manejan la coordinación entre diferentes servicios cuando es necesario, pero nunca contienen lógica de negocio propia. Esta separación permite que el frontend consuma una API clara y consistente sin conocer los detalles internos de la implementación.

La implementación sigue un patrón consistente donde cada controlador recibe sus dependencias a través de inyección de dependencias en el constructor, utilizando siempre **IRepositoryManager** para crear las instancias de servicio necesarias. Por ejemplo, en el caso del controlador de administración de proyectos, el constructor recibe el repository manager y crea una instancia del servicio correspondiente:

```
public AdminProjectController(IRepositoryManager repositoryManager)
{
    _adminPService = new AdminPService(repositoryManager);
}
```

Hay polimorfismo en el controlador de líderes de proyecto para manejar diferentes formatos de exportación. El controlador mantiene múltiples instancias del mismo servicio, cada una configurada con un exportador diferente, permitiendo generar reportes en formato CSV o JSON:

```
public LeaderProjectController(IRepositoryManager repositoryManager)
```

```
{  
    CSVExporter csvExporter = new CSVExporter(repositoryManager);  
    JSONExporter jsonExporter = new JSONExporter(repositoryManager);  
    _LeaderPServiceCSV = new LeaderPService(repositoryManager, csvExporter);  
    _LeaderPServiceJSON = new LeaderPService(repositoryManager, jsonExporter);  
}
```

Beneficios En La Arquitectura

El desacoplamiento entre frontend y backend permite que ambas capas evolucionen independientemente, facilitando cambios en la lógica de negocio sin impactar la interfaz de usuario y viceversa. Y la reutilización de código se maximiza al permitir que múltiples componentes del frontend utilicen la misma funcionalidad a través de los controladores, eliminando duplicación y asegurando consistencia en las operaciones.

Integración y Escalabilidad

Establecen una interfaz estable y bien definida que actúa como contrato entre el frontend y el backend. Esta permite que el sistema escale tanto en funcionalidad y en complejidad sin requerir cambios notorios en la arquitectura existente. Nuevas funcionalidades pueden agregarse fácilmente creando nuevos controladores siguiendo los mismos patrones establecidos, o extendiendo controladores existentes cuando la funcionalidad está relacionada con un dominio ya existente.

Exporter

Implementamos una arquitectura de exportadores usando los patrones de Template Method para exportar datos de proyectos en múltiples formatos. En el caso del obligatorio solo es en JSON y CSV pero lo hicimos de manera que sea extensible para futuros cambios. A continuación explicamos cada una de las partes de nuestro exporter.

IExporter

Creamos una interfaz IExporter que actúa como el contrato base que todos los exportadores deben cumplir. Esta interfaz define un único método Export que recibe una lista de proyectos y retorna un string con el resultado formateado. El propósito principal es abstraer completamente la implementación específica de cada formato, permitiendo que el código cliente trabaje con cualquier tipo de exportador sin conocer los detalles internos de cómo se genera cada formato

ExporterBase (Abstract)

Desarrollamos ExporterBase como una clase abstracta que implementa el patrón Template Method, centralizando toda la lógica común que comparten todos los exportadores. Esta clase maneja la validación de parámetros de entrada verificando que la lista de proyectos no sea nula, aplica el ordenamiento estándar de proyectos por fecha de inicio según los requerimientos del negocio, filtra proyectos nulos para evitar errores, y delega el formateo específico a cada implementación concreta a través de un método abstracto ExportData. De esta manera, cada nuevo exportador automáticamente hereda todas estas funcionalidades comunes.

CSVExporter - Implementación CSV

Implementamos CSVExporter como una clase concreta que hereda de ExporterBase y se especializa en generar archivos en formato CSV. En esta implementación construimos los headers apropiados para las columnas del archivo, manejamos el escape correcto de campos que contienen caracteres especiales como comas, comillas o saltos de línea, utiliza punto y coma como separador para múltiples recursos evitando conflictos con las comas del CSV, y aplicamos el formato de fecha DD/MM/YYYY según las especificaciones. También gestionamos casos especiales como recursos nulos o vacíos para mantener la integridad del formato

JSONExporter - Implementación JSON

Creamos JSONExporter que genera salida en formato JSON estructurado, utilizando DTOs específicos llamados ProjectExportDTO y TaskExportDTO para controlar exactamente la estructura del resultado final. El uso de DTOs específicos nos permite tener un control sobre qué campos se incluyen y cómo se estructuran en el JSON final.

Integración con LeaderPService

Finalmente hicimos el sistema completo de exportadores en LeaderPService mediante inyección de dependencias, recibiendo una instancia de IExporter en el constructor del servicio. El método ExportProjects obtiene los proyectos del usuario actual a través del AdminPService, valida que existan proyectos para exportar, delega toda la lógica de exportación al exportador inyectado, y maneja las excepciones de manera apropiada lanzando UnableToExportProject en caso de error. Esta integración permite configurar

externamente qué tipo de exportador utilizar, ya sea a través del contenedor de inyección de dependencias o mediante patrones de factory.

Ventajas para Extensibilidad

Esta implementación que hicimos tiene muchas ventajas para la extensibilidad. La arquitectura que usamos, combinando los patrones Template Method y Strategy, nos permite agregar nuevos formatos de exportación de manera fácil y sin tocar el código que ya tenemos. Si en el futuro necesitamos agregar, por ejemplo, un PDFExporter, lo único que necesitamos hacer es crear una nueva clase que herede de ExporterBase y que implemente el método ExportData de manera específica para ese formato.

No tenemos que modificar el código existente en otros lugares. El sistema está diseñado de forma que podemos configurar qué tipo de exportador usar sin cambiar nada en el servicio LeaderPService, gracias a la inyección de dependencias.

Además, al mantener toda la lógica común en la clase base ExporterBase, cualquier cambio que hagamos en cómo ordenamos los proyectos o validamos las entradas se va a aplicar automáticamente a todos los exportadores, sin necesidad de actualizar cada uno por separado. Lo que mejora mucho la mantenibilidad.

También hemos hecho que cada exportador sea independiente, lo que significa que podemos probarlos de forma aislada. Cada clase tiene su propio enfoque, lo que facilita las pruebas.

Cambios de DataAccess

Antes

El sistema utilizaba una clase InMemoryDatabase que centralizaba la creación y gestión de todos los repositorios. Esta clase instanciaba todos los repositorios en su constructor, independientemente de si serían utilizados o no, y exponía cada repositorio como una propiedad pública.

Los services recibían la instancia completa de InMemoryDatabase a través de inyección de dependencias y accedían directamente a los repositorios específicos que necesitaban.

```
public AdminPService(InMemoryDatabase database)  
  
{  
  
    _database = database;
```



```
}
```

```
// Acceso directo a repositorios
```

```
var project = _database.projects.GetProject(p => p.Name == name);
```

Esta implementación presentaba varios problemas: se creaban repositorios innecesarios, los services tenían acceso a todos los repositorios (cuando tendrían que tener acceso solamente al que necesitan), y cualquier cambio en la implementación de acceso a datos requería modificaciones en múltiples clases.

Ahora (cambios implementados)

Introducimos una interfaz genérica IRepository<T> que define las operaciones básicas de acceso a datos que todos los repositorios deben implementar: GetAll(), Get(), Add(), Update() y Delete(). Esto nos garantiza que todos los repositorios tengan una interfaz consistente, lo que facilita el mantenimiento del código y estandariza el uso de los repositorios.

También creamos IRepositoryManager, que define qué repositorios están disponibles en el sistema sin exponer los detalles de su implementación. Esta interfaz actúa como un contrato, especificando los repositorios que pueden ser solicitados por los servicios.

```
public interface IRepositoryManager
```

```
{
```

```
    UserRepository UserRepository { get; }
```

```
    ProjectRepository ProjectRepository { get; }
```

```
    NotificationRepository NotificationRepository { get; }
```

```
    // ...otros repositorios
```

```
}
```

RepositoryManager implementa IRepositoryManager y actúa como intermediario entre los services y la capa de acceso a datos. Su responsabilidad principal es gestionar la creación y el ciclo de vida de los repositorios.

La característica más importante de esta implementación es el uso de lazy loading: cada repositorio se crea únicamente cuando es solicitado por primera vez. Esto se logra mediante

propiedades que verifican si el repositorio ya existe en memoria, y si no es así, lo crean utilizando el ApplicationDbContext proporcionado.

```
public UserRepository UserRepository =>  
  
    _userRepository ??= new UserRepository(_context);
```

El RepositoryManager recibe ApplicationDbContext en su constructor y lo utiliza para crear los repositorios según van siendo necesarios. Esto optimiza el uso de memoria y mejora el rendimiento de la aplicación.

Comunicación Service - Data Access

Antes: Service -> InMemoryDatabase -> Repositorio específico

Ahora: Service -> IRepositoryManager -> RepositoryManager -> Repositorio específico

Los services ahora reciben IRepositoryManager en lugar de InMemoryDatabase. Cuando un service necesita acceder a datos, solicita el repositorio correspondiente al manager, quien se encarga de proporcionarlo, creándolo si es necesario.

Migración a Entity Framework

En paralelo, migramos el sistema de almacenamiento desde listas en memoria hacia Entity Framework Core con ApplicationDbContext. Esto prepara nuestra aplicación para trabajar con bases de datos reales y nos permite aprovechar las funcionalidades avanzadas de un ORM.

Configuración de Dependencias

La configuración en Program.cs se simplificó considerablemente:

Configuración anterior:

```
builder.Services.AddSingleton<InMemoryDatabase>();
```

Nueva configuración:

```
builder.Services.AddScoped<IRepositoryManager, RepositoryManager>();
```

Beneficios de los cambios

Los beneficios obtenidos son una mayor flexibilidad de implementación, ya que el desacoplamiento nos permite cambiar la implementación de acceso a datos sin afectar los servicios. Esto nos permite migrar a diferentes tecnologías de persistencia (como SQL Server) modificando únicamente la implementación de RepositoryManager. También logramos una optimización de recursos gracias al lazy loading, que evita la creación innecesaria de repositorios. Si un servicio sólo necesita acceder a datos de usuarios, no se crearán los repositorios de proyectos, notificaciones u otras entidades, ahorrando memoria y tiempo de inicialización.

La mantenibilidad mejorada es otro beneficio clave, ya que los cambios en la capa de acceso a datos están localizados y no se propagan a través de toda la aplicación. Cada componente tiene responsabilidades bien definidas, lo que facilita la comprensión y modificación del código. Además, la arquitectura resultante está preparada para la escalabilidad.

En cuanto a los principios de diseño aplicados, la separación de responsabilidades es más clara, con cada clase enfocándose en una tarea específica. El acoplamiento entre componentes se reduce al depender de abstracciones en lugar de implementaciones concretas, aplicando inversión de dependencias de manera efectiva. También mejoramos la cohesión dentro de cada clase, ya que las responsabilidades están mejor organizadas y relacionadas. Además, la creación de objetos se maneja siguiendo el principio de que quien mejor puede crear un objeto es quien tiene la información necesaria y lo utiliza intensivamente.

El bajo acoplamiento se logra al hacer que los servicios dependan de abstracciones (interfaces) en lugar de implementaciones concretas, lo que permite cambiar tecnologías sin afectar los servicios. La alta cohesión se logra porque cada clase tiene una responsabilidad clara y está enfocada en una tarea específica, lo que mejora la comprensión, modificación y mantenimiento del código.

Análisis de Dependencias

Resolución de conflictos en asignación de recursos a tareas

Análisis de Dependencias

En el desarrollo de nuestro sistema para la resolución de conflictos en la asignación de recursos a tareas, nos enfrentamos a un desafío clave: asegurar que múltiples tareas, que requieren simultáneamente recursos limitados, puedan ser gestionadas sin generar

conflictos. Para ello, definimos un flujo claro de detección y resolución de conflictos, que se basa principalmente entre las clases TaskService y ResourceService.

Clases del Dominio Involucradas

Dentro de nuestro sistema, las principales clases involucradas en la gestión de recursos y resolución de conflictos son:

- TaskService: Esta clase es quien crea tareas y gestiona conflictos entre ellas. Su responsabilidad principal es coordinar la interacción con ResourceService para verificar la disponibilidad de recursos.
- ResourceService: Se encarga de gestionar la disponibilidad de recursos y asignarlos a las tareas. Además, maneja la lógica que permite verificar la disponibilidad de estos recursos en función de la fecha y duración de las tareas.
- Project y Task: Representan el proyecto y las tareas individuales dentro de él. Task mantiene la información sobre los recursos asignados y sus fechas, mientras que Project define el contexto general y agrupa las tareas.
- CpmService: Encargado de calcular el camino crítico después de resolver cualquier conflicto de recursos.
- NotificationService: Gestiona las notificaciones relacionadas con cambios en el camino crítico o en la asignación de recursos.

Detección y Resolución de Conflictos

El proceso de resolución de conflictos inicia cuando se intenta crear una nueva tarea. En este punto, el método AddTask de TaskService llama a GetNextDateAvailable para encontrar una fecha en la que los recursos necesarios estén disponibles. Esta verificación se realiza a través de los métodos IsAvailable y NextDateAvailable de ResourceService, los cuales verifican si existe algún conflicto con las tareas ya programadas.

Si se detecta un conflicto y el parámetro solve es falso, el sistema lanza una excepción ResourceNotAvailableException. Si solve es verdadero, el sistema busca automáticamente una nueva fecha disponible. Este proceso se realiza mediante una búsqueda secuencial día por día hasta encontrar un espacio donde todos los recursos estén disponibles. Lo hicimos como fuerza bruta ya que consideramos que no van a haber casos de usos tan masivos.

Ejemplo de Resolución de Conflictos:

Se intenta asignar un recurso a una nueva tarea, pero el recurso ya está comprometido en otra tarea durante el mismo período. En este caso, el método IsAvailable verifica la superposición de las fechas, y si el recurso no está disponible, el sistema reprograma la tarea en una fecha futura, buscando la próxima fecha libre, lo que evita que las tareas se solapen.

Además, ResourceService tiene la responsabilidad de gestionar las dependencias automáticas entre tareas que utilizan los mismos recursos. Por ejemplo, si dos tareas comparten un recurso no concurrente, ResourceService crea una dependencia implícita entre ellas, garantizando que una tarea no comience antes de que termine la otra, evitando así posibles conflictos.

Acoplamiento del Sistema

La arquitectura de nuestro sistema en este caso tiene un alto acoplamiento entre algunas clases. TaskService tiene una dependencia crítica de ResourceService, ya que no solo invoca sus métodos, sino que también depende de su lógica interna y estructuras de datos. Este acoplamiento por control y datos hace que el sistema sea más difícil de modificar y probar.

Por otro lado, TaskService también depende de RepositoryManager para acceder a los repositorios de datos. Las dependencias hacia clases como CpmService y NotificationService son más ligeras, ya que solo se utilizan para invocar sus servicios sin necesidad de comprender sus implementaciones internas.

Cohesión del Sistema

La cohesión en nuestro sistema es en general buena, aunque con algunas áreas de mejora. Por ejemplo, TaskService tiene una alta cohesión funcional, ya que todas sus operaciones están orientadas a la gestión de tareas. Sin embargo, el uso del método RecalculateCriticalPath dentro de AddTask introduce una responsabilidad adicional que puede restar cohesión.

Por su parte, ResourceService tiene una fuerte cohesión, ya que todos sus métodos están centrados en la gestión de recursos. Por otro lado, el método updateResourceDependencies, que maneja tanto la lógica de recursos como las dependencias entre tareas, podría beneficiarse de una separación en un servicio especializado para la gestión de dependencias.

Posibles mejoras

Aunque el sistema implementa efectivamente la resolución de conflictos, la dependencia directa entre TaskService y ResourceService presenta problemas para el mantenimiento y la evolución del sistema. Podríamos haber introducido interfaces para los servicios de resolución de conflictos y una mayor separación de responsabilidades. Esto permitiría reducir el acoplamiento y mejorar la testabilidad, sin comprometer la funcionalidad actual.

Datos de Prueba:

Para la creación de los datos de prueba utilizamos únicamente la UI, los creamos directamente desde ahí para asegurarnos de un buen funcionamiento de todo antes de

exportarlos. Luego con DBeaver los exportamos a un script que se puede correr para insertar nuevamente los datos.

Los datos de prueba se componen en nuestro caso de 21 usuarios, todos registrados con la misma contraseña Password123# para facilitar el ingreso.

Incluimos en los usuarios: un usuario sin roles, un usuario con todos los roles, un usuario con solo rol de administrador de sistema, cuatro administradores de proyecto, un usuario administrador de proyecto y líder de proyecto, un usuario administrador de proyecto y miembro, un usuario líder de proyecto y miembro, tres líderes de proyecto y 8 miembros de proyecto (4 normales y 4 con distinciones, por ejemplo uno con todas las tareas, otro sin tareas asignadas, etc).

Además Creamos 10 proyectos para probar distintas cosas, por ejemplo un proyecto que tiene como Líder de proyecto al administrador, otro que tiene como miembro al Líder, otro que tiene como administrador, líder y miembro al mismo usuario, a otro no le creamos tarea... Intentamos abarcar todos los casos que se nos ocurrieron.

También creamos 22 tareas, las cuales están repartidas entre los proyectos. Además 10 de esas tareas cuentan con dependencias.

También creamos 7 recursos, 3 de uso global, es decir para todos los proyectos, y 4 exclusivos de un solo proyecto. También hay 2 que tienen el uso concurrente habilitado.

Ejemplo de tarea con recursos:

Para crear una tarea que deba forzar la asignación de recurso se podría hacer ingresando como: AdminLeaderP@gmail.com y con la contraseña Password123#.

Luego en el Proyecto: Project With Same Admin And Leader crear una nueva tarea con fecha de inicio: 18 de junio de 2025, que no tenga dependencias y que utilice el recurso: Resource 3. Al hacer eso se desplegará el menú que nos permitirá forzar la creación o cancelarla, en caso de que la cancelemos podremos modificar la tarea desde la propia página para crearla nuevamente en otra fecha o utilizando otro recurso. Si decidimos forzar la creación, nuestro sistema moverá internamente la tarea hasta la fecha mas proxima que este disponible el recurso para la duración de la tarea

Posibles Mejoras

Una vez que terminamos con todo el proyecto , nos dimos cuenta que hay algunos factores mejorables.

El primero es que tenemos una page dentro de AdminPPages llamada ProjectList la cual tiene aproximadamente 1800 líneas de código. En esta hacemos todas las operaciones

relacionadas a los proyectos. Podríamos haber dividido esta page por funcionalidades de los proyectos para que quede de menos líneas y mas entendible. Esto mismo lo intentamos hacer pero teníamos problemas para comunicar las pages entre sí, por ejemplo si hacíamos una page para añadir la tarea al proyecto y una que administre esa page, las variables que utilizabamos en una no la podíamos hacer llegar a la otra; Estuvimos un tiempo intentando solucionarlo pero decidimos avanzar en todas las funcionalidades, terminar el proyecto y entregar algo que funcione y no perder el tiempo con estos. El código viola SRP, la Alta Cohesión y Bajo Acoplamiento y los principios de Clean Code de funciones pequeñas al concentrar 1800 líneas con todas las operaciones de proyectos en una sola página Blazor. Para mejorarlo se debe implementar servicios de estado (State Services con dependency injection), usar EventCallback y CascadingParameter para comunicación entre componentes, y dividir en componentes especializados: ProjectListComponent, ProjectFormComponent, ProjectDetailsComponent, etc., registrando los servicios en Program.cs para compartir estado entre páginas y lograr responsabilidades únicas.

Otra cosa que podríamos haber mejorado es el acoplamiento que tiene User.cs en el Domain. Al hacer el Uml nos dimos cuenta que esta clase en concreto tiene muchas relaciones con otras, lo cual genera un acoplamiento alto. Este se vincula con Rol, Task, Notification y Project. Para mejorar el acoplamiento podríamos haber implementado clases intermedias, por ejemplo hacer una clase intermedia llamada NotificationUser que gestione la lista de notificaciones del usuario como los métodos de agregar, eliminar, etc. No lo hicimos porque nos quedaban muy pocos días para la entrega y esto nos cambiaba bastante toda la lógica de los servicios y otras partes de nuestro programa.

Anexo

Cálculo De Ruta Crítica: TDD

Cómo hicimos para las demás partes del proyecto, implementamos el CPMSERVICE utilizando TDD , regido por las etapas red, green y refactor. Esto en la clase CPMSERVICETest.

Pruebas de Excepciones:

Las hicimos para que el sistema maneje correctamente los casos donde los datos de entrada son inválidos o no cumplen con lo pedido en la letra. Se validan las excepciones que deben lanzarse.

Un ejemplo es CalculateCriticalPath_ShouldThrowException_WhenTasksListIsNullOrEmpty verifica que el sistema lance una NullTaskListException cuando se pasa una lista de tareas nula al método CalculateCriticalPath.

Pruebas de Cálculo de Fechas Tempranas:

Estas pruebas verifican que el cálculo de las fechas de inicio y finalización tempranas sea correcto. También la verificación de tareas sin dependencias (que deberían comenzar en la fecha esperada) y tareas que dependen de otras (que deberían iniciar después de las tareas previas).

Por ejemplo:

CalculateCriticalPath_ShouldCalculateEarlyDates_ForTaskWithPredecessors valida que las fechas de inicio y finalización se calculen correctamente para una tarea que depende de otra. La tarea C debe empezar después de la tarea A, y la fecha de finalización de la tarea C debe ser la fecha de inicio de C + su duración.

Pruebas de Cálculo de Fechas Tardías y Slack:

Estas verifican que el sistema calcule las fechas tardías de inicio y finalización correctamente, como el cálculo del Slack de cada tarea. Las tareas críticas deben tener un Slack de 0, y las tareas no críticas deben tener un margen de retraso.

Un ejemplo es CalculateCriticalPath_ShouldCalculateLateDates_AndSlack valida que las tareas con dependencias calculen correctamente sus fechas tardías, y además, verifica que el Slack de las tareas críticas sea 0, indicando que no hay margen para retraso. Si una tarea tiene Slack mayor a 0, no es crítica.

Pruebas de Ruta Crítica:

Estas pruebas validan que el algoritmo forme correctamente la ruta crítica, las tareas que su retraso afectará la duración total del proyecto. Hicimos casos de prueba con diferentes estructuras, incluyendo tareas paralelas y dependencias complejas.

Un ejemplo es

`CalculateCriticalPath_ShouldIdentifyCriticalPath_WithComplexDependencies` verifica que la ruta crítica se cree correctamente en un conjunto de tareas con dependencias complejas. La función debe identificar qué tareas son críticas y cuáles no, basándose en las fechas de inicio y finalización calculadas y en las relaciones entre tareas.

Pruebas de Duración del Proyecto:

Aseguran que la duración total del proyecto se calcule correctamente. Esto incluye sumar las duraciones de las tareas y tener en cuenta las dependencias entre ellas para determinar el tiempo total requerido para completar el proyecto.

Por ejemplo, la prueba `CalculateProjectDuration_HandlesEmptyList` valida que si solo hay una tarea en el proyecto, su duración sea correctamente calculada como el tiempo de la tarea misma. Si hay múltiples tareas, la duración total debe considerar las dependencias y el orden de ejecución de las tareas.

Dependencias Externas

DHTMLX Gantt

Utilizamos la librería DHTMLX Gantt para hacer el diagrama de Gantt. La descargamos e incluimos al proyecto para hacer que funcione de forma local, permitiendo así acceder al diagrama sin conexión a internet.

Bootstrap

Usamos bootstrap para la parte de front-end, principalmente para hacer la página responsive y que se vea bien en todos los dispositivos. También la utilizamos para la parte de los iconos, de manera que la página se vea más estética, amena y entendible para el usuario final.

Full Calendar

Se implementó un panel de calendario de recursos usando la librería Full Calendar para mostrar de manera intuitiva los períodos en que cada recurso está ocupado y su próxima fecha disponible.

La configuración de FullCalendar se realiza en App.razor incluyéndose vía CDN (fullcalendar@6.1.8/index.global.min.js y su hoja de estilos). Se define una función global `initResourceCalendar(events)` en la que se inicializa o destruye el calendario existente, se establecen opciones de vista (`dayGridMonth`), cabecera, altura y se pasan los eventos (ocupación y disponibilidad).

En la página `ResourcePanel.razor` se carga primero la lista de proyectos para el usuario actual, luego, al elegir un proyecto, se obtienen sus recursos mediante el controlador. Al seleccionar un recurso, se consulta cuándo está ocupado y su próxima fecha libre, generando un array de objetos con propiedades `title`, `start`, `end` y `color`. Ese array se almacena en `_pendingEvents` y se dispara la inicialización del calendario.

Gracias al uso de FullCalendar, los administradores pueden detectar rápidamente solapamientos de uso y huecos libres con un simple vistazo al calendario, optimizando la asignación de recursos.

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.10.5/font/bootstrap-icons.css">
<link href="https://cdn.jsdelivr.net/npm/bootstrap-icons@1.11.1/font/bootstrap-icons.css">
```

Uso de IA (Primer Entrega)

Usamos inteligencia artificial para corroborar alguna de las funcionalidades implementadas, para lo que más nos ayudó fue para entender cómo funcionaba un CPM y para implementar partes como el password manager:

<https://chatgpt.com/share/682603dc-50d0-800b-8cb7-47df4307b627>

<https://chatgpt.com/share/e/68260df7-d694-8007-982c-f629f676c058>

<https://chatgpt.com/share/68264e49-f7e8-800e-a717-f915c3c0a145>

Para la parte de CPM primero lo hablamos con chat gpt para entender las bases del mismo y cuales son sus funciones principales pero quien más nos dio una mano con la implementación fue la ia Claude, nos pusimos a buscar el chat en el que nos ayudó con el CPM pero no lo encontramos.

En la parte del diagrama de Gantt se puede ver cómo lo utilizamos para buscar la librería e implementarlo en nuestra app.

Uso de IA (Segunda Entrega)

Usamos inteligencia artificial para corroborar alguna de las funcionalidades implementadas, para lo que más nos ayudó fue para un mapeo de como hacer el exportador (ya teníamos una idea pero buscábamos reforzar un poco más)y para implementar partes como el calendario en el panel de recursos

Búsqueda de librería e implementación del calendario:

<https://chatgpt.com/share/684b66f8-054c-800e-b6c3-e2e394eb40de>

Chat para implementar exportador:

<https://chatgpt.com/share/684b64a8-d670-8005-8615-d03c3eb6bb31>

Foto coverage de pruebas

Symbol	Coverage (%) ▾	Uncovered/Total Stmts.
✓ Total	94%	459/7334
> Service.Test	97%	110/3221
> DataAccess.Tests	95%	29/550
> Domain.Test	94%	29/513
> Domain	91%	33/376
> DataAccess	91%	43/456
> Service	90%	215/2218

Como se puede ver llegamos a un 90% de coverage en todas los packages. Es importante excluir las migraciones de DataAccess ya que para estas no hay tests.

También no hicimos tests para los Controllers ya que estos funcionan como intermediarios entre Services y la UI, los métodos que utilizan los controllers ya fueron testeados en los tests de service.